Attitude Dynamics and Control of a Nano-Satellite Orbiting Mars

Gunnar Montseny Gens*
University of Colorado Boulder, Boulder, Colorado 80303

This work studies the attitude dynamics and control of a nano-satellite in low Mars orbit (LMO) subject to multiple pointing requirements, including nadir, Sun, and communication with mother spacecraft in a geosynchronous Mars orbit (GMO). The reference attitudes for these requirements are constructed from orbital geometry, assuming circular two-body motion. The spacecraft is modeled as a rigid body and controlled through a proportional-derivative (PD) law to drive the spacecraft toward different reference attitudes. The spacecraft attitude motion is numerically integrated through a fixed-step 4th order Runge–Kutta routine, leading to a full mission scenario simulation. Simulation results validate the modeling and control framework across all mission scenarios.

I. Introduction

Spacecraft operating in planetary orbits are often required to satisfy multiple pointing constraints that vary throughout the mission. In particular, observation, communication, and power generation objectives impose distinct attitude requirements that cannot be achieved simultaneously, requiring the spacecraft to transition between different pointing modes over the course of an orbit.

In this work, we consider a nano-satellite in a circular low Mars orbit (LMO) performing scientific observations. The collected data must then be transmitted to the mother (relay) spacecraft in a circular geosynchronous Mars grbit (GMO). Further, to keep the batteries from draining all the way, periodically the satellite must point its solar panels at the Sun to recharge. Thus, three mission goals must be considered by the nano-satellite:

1) Point science instruments toward the Mars nadir direction.
2) Point communication hardware at the mother satellite.
3) Point the solar arrays at the Sun.

The high-level goal of this paper is to construct these reference frames and design an attitude control law to achieve these pointing objectives. In order to satisfy the mission requirements, we drive the spacecraft's body frame towards various reference frames that correspond to desired attitudes. Note that the spacecraft is modeled as a rigid body with perfect state knowledge and no external disturbances.

II. Mission Overview

This section presents all the details of the mission to the reader. The details of the different mission pointing scenarios are specified, along with the frames used in this work. In addition, we provide more information about the initial state of the nano-satellite.

A. Frame Definition

Let $\mathcal{N} = \{\hat{n}_1, \hat{n}_2, \hat{n}_3\}$ be the inertial frame with its origin set at the center of mass of Mars, with the corresponding unit vectors shown in Figure 1. Also consider body-frame $\mathcal{B} = \{\hat{b}_1, \hat{b}_2, \hat{b}_3\}$, with origin at the center of mass of the LMO spacecraft and depicted in Figure 2. The LMO spacecraft is equipped with an antenna to communicate with the mothercraft (aligned with $-\hat{b}_1$), a sensor to observe Mars (aligned with $\hat{b}_1$), and solar panels for power (panel normal is along $\hat{b}_3$).

Further, a general circular orbit frame $\mathcal{O} = \{\hat{\imath}_r, \hat{\imath}_\theta, \hat{\imath}_h\}$ is used for initial derivations. $\hat{\imath}_r$ points to the spacecraft, $\hat{\imath}_h$ is the direction of the angular momentum vector, and $\hat{\imath}_\theta = \hat{\imath}_h \times \hat{\imath}_r$. However, a Hill frame $\mathcal{H} = \{\hat{\imath}_r, \hat{\imath}_\theta, \hat{\imath}_h\}$ is used throughout the paper to specifically describe a circular LMO. Note that the Hill frame is given through the (3-1-3) Euler angle set (Ω, i, θ). See Figure 1 for an illustration of the Hill frame with respect to the inertial frame.

*Ann and H.J. Smead Department of Aerospace Engineering Sciences, University of Colorado Boulder, 3775 Discovery Drive, Boulder, CO 80303, USA

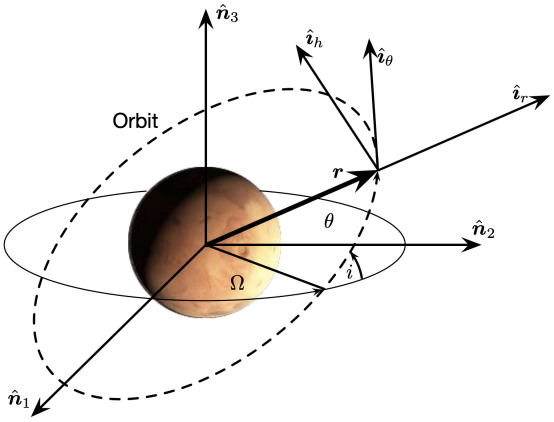

Fig. 1 Illustration of Inertial Frame $\mathcal{N}$ and Hill frame $\mathcal{H} = \{\hat{\imath}_r, \hat{\imath}_\theta, \hat{\imath}_h\}$.

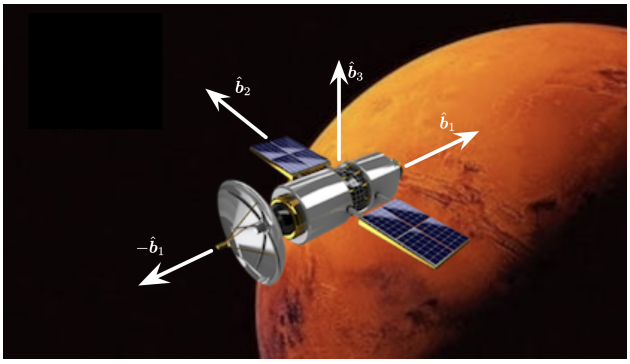

Fig. 2 Illustration of the spacecraft component directions, and definition of the body-frame $\mathcal{B}$..

In addition, three body reference frames are considered and constructed in Section III: Sun-pointing reference frame $\mathcal{R}_s$, nadir-pointing reference frame $\mathcal{R}_n$, and GMO-pointing reference frame $\mathcal{R}_c$.

B. Mission Orbit Overview

As mentioned, in this study we assume that the orbits of both spacecrafts are circular. Consequently, the orbits have a constant radius and a constant angular rate. Let the Mars gravity constant be $\mu_{Mars} = 42828.3$ km^3/s^2. Then, the constant rate of a circular orbit with radius r can be calculated with the following expression:

$$\dot{\theta}(r) = \sqrt{\frac{\mu_{Mars}}{r^3}}. \tag{1}$$

Table 1 presents the radius, angular rates, and initial (3-1-3) Euler angles for both orbits.

Table 1 Orbit Overview Details

Spacecraft	r (km)	$\dot{\theta}$ (rad/s)	$\Omega(t_0)$ (deg)	$i(t_0)$ (deg)	$\theta(t_0)$ (deg)
LMO	3796.19	0.000884797	20	30	60
GMO	20424.2	0.0000709003	0	0	250

C. Mission Pointing Scenarios

This section defines the various attitude pointing modes for the spacecraft. For simplification, it can be assumed that the sun is an infinite distance away and always in the $\hat{n}_2$ direction as illustrated in Figure 3. During the mission, the control law must detumble the satellite and either point its antenna ($-\hat{b}_1$) toward the GMO mother satellite, its sensor ($\hat{b}_1$) at Mars (nadir direction), or its solar panels ($\hat{b}_3$) at the Sun. Due to the high power demands of the telescope, the spacecraft must point its solar panels toward the Sun whenever it is on the sunlit side of Mars (i.e., positive satellite $\hat{n}_2$ position coordinate). In other words, when pointing at the Sun, the spacecraft must align its solar panel axis $\hat{b}_3$ with the $\hat{n}_2$ direction.

When the spacecraft is on the shaded side of Mars (i.e., over the hemisphere with negative $\hat{n}_2$ position coordinate), it must enter either communication or science mode. In science mode, the science platform axis $\hat{b}_1$ must point toward the center of Mars, i.e., in the nadir direction. The communication mode requires that the angular separation between the LMO and GMO position vectors be less than 35°. In this mode, the nano-satellite communication platform axis $-\hat{b}_1$ must point toward the GMO mother spacecraft. Table 2 summarizes these pointing requirements.

Table 2 Spacecraft Pointing Scenario Summary

Orbital Situation	Primary Pointing Scenario Goals
Spacecraft on sunlit Mars side	Point solar axis $\hat{b}_3$ at Sun
Spacecraft not on sunlit Mars side and GMO visible	Point antenna axis $-\hat{b}_1$ at mothercraft
Spacecraft not on sunlit Mars side and GMO not visible	Point sensor axis $\hat{b}_1$ along Mars nadir direction

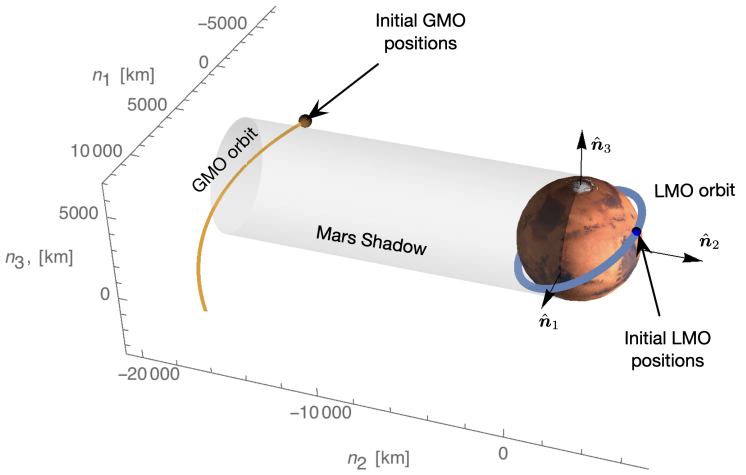

Fig. 3 Mission Orbit Scenario Illustration.

D. Spacecraft Attitude States

The spacecraft's initial attitude is given through the following Modified Rodrigues Parameter (MRP) set:

$$\vec{\sigma}_{\mathcal{B}/\mathcal{N}}(t_0) = \begin{bmatrix} 0.3 \\ -0.4 \\ 0.5 \end{bmatrix}. \tag{2}$$

The initial body angular velocity with respect to the inertial frame is given by:

$$^{\mathcal{B}}\vec{\omega}_{\mathcal{B}/\mathcal{N}}(t_0) = {}^{\mathcal{B}}\begin{bmatrix} 1.00 \\ 1.75 \\ -2.20 \end{bmatrix} \text{deg/s}. \tag{3}$$

The rigid LMO spacecraft inertia tensor is given by:

$$^{\mathcal{B}}[I] = {}^{\mathcal{B}}\begin{bmatrix} 10 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 7.5 \end{bmatrix} \text{kg m}^2. \tag{4}$$

Note that $^{\mathcal{B}}[I]$ is a diagonal matrix, indicating that the the three axes in body frame $\mathcal{B}$ are principal axes of our rigid LMO spacecraft.

III. Modeling and Control Framework

A. Orbit Simulation

The position vector on a circular orbit of radius r can be described through frame $\mathcal{O}$ as $^{\mathcal{O}}\vec{r} = r\hat{\imath}_r$. To obtain the inertial velocity, we apply the Transport Theorem:

$$\dot{\vec{r}} = \frac{^{\mathcal{N}}d}{dt}\left(^{\mathcal{O}}\vec{r}\right) = \frac{^{\mathcal{O}}d}{dt}\left(^{\mathcal{O}}\vec{r}\right) + {}^{\mathcal{O}}\vec{\omega}_{\mathcal{O}/\mathcal{N}} \times {}^{\mathcal{O}}\vec{r} = \dot{r}\hat{\imath}_r + {}^{\mathcal{O}}\vec{\omega}_{\mathcal{O}/\mathcal{N}} \times {}^{\mathcal{O}}\vec{r} \tag{5}$$

By definition of a circular orbit, we know that $^{\mathcal{O}}\vec{\omega}_{\mathcal{O}/\mathcal{N}} = \dot{\theta}\hat{\imath}_h$. In addition, since the $\mathcal{O}$ frame follows a circular orbit, $\dot{r} = 0$, and the inertial velocity can be written as

$$^{\mathcal{O}}\dot{\vec{r}} = r\dot{\theta}\hat{\imath}_\theta = {}^{\mathcal{O}}\begin{bmatrix} 0 \\ r\dot{\theta} \\ 0 \end{bmatrix}. \tag{6}$$

To express the vectors $^{\mathcal{O}}\vec{r}$ and $^{\mathcal{O}}\dot{\vec{r}}$ in the $\mathcal{N}$ frame, we left-multiply them by the correspoding Direction Cosine Matrix (DCM):

$$^{\mathcal{N}}\vec{r}\,(t) = [\mathcal{N}\mathcal{O}]\Big|_t \, ^{\mathcal{O}}\vec{r}\,(t), \qquad ^{\mathcal{N}}\dot{\vec{r}}\,(t) = [\mathcal{N}\mathcal{O}]\Big|_t \, ^{\mathcal{O}}\dot{\vec{r}}\,(t), \tag{7}$$

where $[\mathcal{O}\mathcal{N}]$ is the DCM that maps the $\mathcal{N}$ frame to the $\mathcal{O}$ frame. It can be defined with the 3-1-3 Euler angles $(\Omega, i, \theta(t))$:

$$[\mathcal{O}\mathcal{N}]\Big|_t = \begin{bmatrix} \cos\theta(t)\cos\Omega - \sin\theta(t)\cos i\sin\Omega & \cos\theta(t)\sin\Omega + \sin\theta(t)\cos i\cos\Omega & \sin\theta(t)\sin i \\ -\sin\theta(t)\cos\Omega - \cos\theta(t)\cos i\sin\Omega & -\sin\theta(t)\sin\Omega + \cos\theta(t)\cos i\cos\Omega & \cos\theta(t)\sin i \\ \sin i\sin\Omega & -\sin i\cos\Omega & \cos i \end{bmatrix}. \tag{8}$$

Note that we assume that both spacecraft are in a circular orbit, meaning that the true anomaly $\theta(t)$ can be expressed as:

$$\theta(t) = \theta(t_0) + \dot{\theta}(t - t_0), \tag{9}$$

where $\dot{\theta}$ is constant, and $\theta(t_0)$ is the true anomaly of the chosen orbit at time t_0. These values are provided for both the LMO and GMO case in Table 1.

B. Orbit Frame Orientation

Consider the Hill frame $\mathcal{H} = \{\hat{\imath}_r, \hat{\imath}_\theta, \hat{\imath}_h\}$, the orbit frame of the LMO satellite. We define these vectors as

$$\hat{\imath}_r = \frac{\vec{r}_{\text{LMO}}}{\|\vec{r}_{\text{LMO}}\|}, \qquad \hat{\imath}_\theta = \hat{\imath}_h \times \hat{\imath}_r, \qquad \hat{\imath}_h = \frac{\vec{r}_{\text{LMO}} \times \dot{\vec{r}}_{\text{LMO}}}{\|\vec{r}_{\text{LMO}} \times \dot{\vec{r}}_{\text{LMO}}\|},$$

where r_{LMO} can be found in Table 1, and $\dot{r}_{\text{LMO}} = r_{\text{LMO}}\dot{\theta}_{\text{LMO}}$. To calculate the DCM $[\mathcal{H}N]\Big|_t$, we use Eq. (8) with $(\Omega, i, \theta) = (\Omega_{\text{LMO}}, i_{\text{LMO}}, \theta_{\text{LMO}}(t))$, where Ω_{LMO} and i_{LMO} can be found in Table 1, and $\theta_{\text{LMO}}(t)$ can be computed through Eq. (9).

C. Sun-Pointing Frame Orientation

To point the spacecraft solar panels axis $\hat{b}_3$ at the sun, the reference frame $\mathcal{R}_s = \{\hat{r}_1, \hat{r}_2, \hat{r}_3\}$ must be chosen such that $\hat{r}_3$ axis points in the sun direction ($\hat{n}_2$ in this scenario). Further, we assume that the first axis $\hat{r}_1$ points in the $-\hat{n}_1$ direction. Consequently, we have that $\hat{r}_1 = -\hat{n}_1$, and $\hat{r}_3 = \hat{n}_2$. To ensure that $\mathcal{R}_s$ is a right-handed frame, we find the last unit vector of the frame through $\hat{r}_2 = \hat{r}_3 \times \hat{r}_1 = \hat{n}_3$.

By definition, the DCM that maps vectors from the inertial frame $\mathcal{N}$ to the Sun-pointing frame $\mathcal{R}_s$ is:

$$[\mathcal{R}_s N] = \begin{bmatrix} {}^N\hat{r}_1^T \\ {}^N\hat{r}_2^T \\ {}^N\hat{r}_3^T \end{bmatrix} = \begin{bmatrix} -{}^N\hat{n}_1^T \\ {}^N\hat{n}_3^T \\ {}^N\hat{n}_2^T \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}. \tag{10}$$

Note that ${}^N\vec{\omega}_{\mathcal{R}_s/N} = \vec{0}$ because the reference frame $\mathcal{R}_s$ is constant in time with respect to the inertial frame $\mathcal{N}$.

D. Nadir-Pointing Reference Frame Orientation

To point the spacecraft sensor platform axis $\hat{b}_1$ towards the center of Mars or nadir direction, the reference frame $\mathcal{R}_n = \{\hat{r}_1, \hat{r}_2, \hat{r}_3\}$ must be chosen such that $\hat{r}_1$ axis points towards the planet ($\hat{r}_1 = -\hat{\imath}_r$). Further, we assume that the second axis $\hat{r}_2$ points in the velocity direction $\hat{\imath}_\theta$. To ensure that $\mathcal{R}_n$ is a right-handed frame, we find the last unit vector of the frame through $\hat{r}_3 = \hat{r}_1 \times \hat{r}_2 = -\hat{\imath}_h$. The DCM $[\mathcal{R}_n\mathcal{H}]$ can be calculated similarly to last section:

$$[\mathcal{R}_n\mathcal{H}] = \begin{bmatrix} {}^\mathcal{H}\hat{r}_1^T \\ {}^\mathcal{H}\hat{r}_2^T \\ {}^\mathcal{H}\hat{r}_3^T \end{bmatrix} = \begin{bmatrix} -{}^\mathcal{H}\hat{\imath}_r^T \\ {}^\mathcal{H}\hat{\imath}_\theta^T \\ -{}^\mathcal{H}\hat{\imath}_h^T \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & -1 \end{bmatrix}. \tag{11}$$

Through properties of DCMs, we know that:

$$[R_n N] = [R_n\mathcal{H}][\mathcal{H}N], \tag{12}$$

The angular velocity of the reference frame $\mathcal{R}_n$ with respect to the inertial frame can be calculated through the following expression:

$${}^\mathcal{H}\vec{\omega}_{\mathcal{R}_n/N} = {}^\mathcal{H}\vec{\omega}_{\mathcal{R}_n/\mathcal{H}} + {}^\mathcal{H}\vec{\omega}_{\mathcal{H}/N} = \dot{\theta}_{\text{LMO}}\hat{\imath}_h. \tag{13}$$

Lastly, we multiply the angular velocity vector by $[N\mathcal{H}]$ to obtain the angular velocity expressed with the inertial frame:

$${}^\mathcal{N}\vec{\omega}_{\mathcal{R}_n/N} = [N\mathcal{H}] \, {}^\mathcal{H}\begin{bmatrix} 0 \\ 0 \\ \dot{\theta}_{\text{LMO}} \end{bmatrix}. \tag{14}$$

It is important to note that both $[\mathcal{R}_n\mathcal{H}]$ and $\|{}^\mathcal{N}\vec{\omega}_{\mathcal{R}_n/N}\|$ are constant in time.

E. GMO-Pointing Reference Frame Orientation

To point the nano-satellite communication platform axis $-\hat{b}_1$ towards the GMO mother spacecraft, the communication reference frame $\mathcal{R}_c = \{\hat{r}_1, \hat{r}_2, \hat{r}_3\}$ must be chosen such that $-\hat{r}_1$ axis points towards the GMO satellite location. We define a relative position vector $\Delta\vec{r}(t) = \vec{r}_{\text{GMO}}(t) - \vec{r}_{\text{LMO}}(t)$, such that $\hat{r}_1 = -\Delta\vec{r}/|\Delta\vec{r}|$. Further, to fully define a three-dimensional reference frame, assume the second axis is defined as

$$\hat{r}_2 = \frac{\Delta\vec{r} \times \hat{n}_3}{|\Delta\vec{r} \times \hat{n}_3|} \tag{15}$$

while the third is then defined as $\hat{r}_3 = \hat{r}_1 \times \hat{r}_2$. The DCM $[\mathcal{R}_c\mathcal{N}]$ can be calculated with the following expression:

$$[\mathcal{R}_c\mathcal{N}] = \begin{bmatrix} {}^{\mathcal{N}}\hat{r}_1^T \\ {}^{\mathcal{N}}\hat{r}_2^T \\ {}^{\mathcal{N}}\hat{r}_3^T \end{bmatrix}, \tag{16}$$

which is not constant in time. As a result, to calculate the angular velocity of this reference frame with respect to the inertial frame, we consider the differential equation for $[\mathcal{R}_c\mathcal{N}]$:

$${}^{\mathcal{R}_c}[\tilde{\omega}_{\mathcal{R}_c/\mathcal{N}}]\Big|_t = -\frac{d}{dt}\left([\mathcal{R}_c\mathcal{N}]\right)\Big|_t [\mathcal{R}_c\mathcal{N}]^T\Big|_t, \tag{17}$$

where $[\tilde{\omega}]$ is the skew-operator of vector $\vec{\omega}$. We numerically calculate the derivative of $[\mathcal{R}_c\mathcal{N}]$ through central differences:

$$\frac{d}{dt}\left([\mathcal{R}_c\mathcal{N}]\right)\Big|_t \approx \frac{[\mathcal{R}_c\mathcal{N}]\Big|_{t+\Delta t} - [\mathcal{R}_c\mathcal{N}]\Big|_{t-\Delta t}}{2\Delta t} \tag{18}$$

Lastly, we extract the necessary components from the skew-operator:

$${}^{\mathcal{N}}\vec{\omega}_{\mathcal{R}_c/N}(t) = [\mathcal{R}_c\mathcal{N}]^T\Big|_t \begin{bmatrix} {}^{\mathcal{R}_c}[\tilde{\omega}_{\mathcal{R}_c/\mathcal{N}}]_{32}\Big|_t \\ {}^{\mathcal{R}_c}[\tilde{\omega}_{\mathcal{R}_c/\mathcal{N}}]_{13}\Big|_t \\ {}^{\mathcal{R}_c}[\tilde{\omega}_{\mathcal{R}_c/\mathcal{N}}]_{21}\Big|_t \end{bmatrix}. \tag{19}$$

F. Attitude Error Evaluation

For our attitude feedback control algorithm we need to compute the attitude and angular velocity tracking errors of current body frame $\mathcal{B}$ relative to a reference frame $\mathcal{R}$ of interest. For our mission, $\mathcal{R} \in \{\mathcal{R}_s, \mathcal{R}_n, \mathcal{R}_c\}$. We assume that the known variables are the attitude states $\sigma_{\mathcal{B}/\mathcal{N}}(t)$, ${}^{\mathcal{B}}\vec{\omega}_{\mathcal{B}/\mathcal{N}}(t)$, as well as the current reference frame DCM $[\mathcal{R}\mathcal{N}]|_t$ and current reference frame rate ${}^{\mathcal{N}}\vec{\omega}_{\mathcal{R}/\mathcal{N}}(t)$. With these variables, we develop a formulation to compute the associated tracking errors $\sigma_{\mathcal{B}/\mathcal{R}}(t)$,${}^{\mathcal{B}}\vec{\omega}_{\mathcal{B}/\mathcal{R}}(t)$. The first step is to calculate the DCM between the inertial frame and the current body frame through the following expression:

$$[\mathcal{B}\mathcal{N}]\Big|_t = [I_{3\times3}] + \frac{8[\tilde{\sigma}_{\mathcal{B}/\mathcal{N}}]^2 - 4\left(1 - \sigma_{\mathcal{B}/\mathcal{N}}^2\right)[\tilde{\sigma}_{\mathcal{B}/\mathcal{N}}]}{\left(1 + \sigma_{\mathcal{B}/\mathcal{N}}^2\right)^2} \tag{20}$$

With this DCM, we can now calculate the angular velocity tracking error:

$${}^{\mathcal{B}}\vec{\omega}_{\mathcal{B}/\mathcal{R}}(t) = {}^{\mathcal{B}}\vec{\omega}_{\mathcal{B}/\mathcal{N}}(t) + {}^{\mathcal{B}}\vec{\omega}_{\mathcal{N}/\mathcal{R}}(t) = {}^{\mathcal{B}}\vec{\omega}_{\mathcal{B}/\mathcal{N}}(t) - [\mathcal{B}\mathcal{N}]\Big|_t {}^{\mathcal{N}}\vec{\omega}_{\mathcal{R}/\mathcal{N}}(t), \tag{21}$$

To calculate the associated attitude tracking error, we have to compute the DCM between the reference frame $\mathcal{R}$ and body frame $\mathcal{B}$:

$$[\mathcal{B}\mathcal{R}]\Big|_t = [\mathcal{B}\mathcal{N}]\Big|_t [\mathcal{R}\mathcal{N}]^T\Big|_t. \tag{22}$$

Lastly, we calculate the attitude tracking error through C2MRP, a function that transforms DCM into MRPs (explanation of the function is provided below):

$$\vec{\sigma}_{\mathcal{B}/\mathcal{R}}(t) = \text{C2MRP}\left([\mathcal{BR}]\Big|_t\right). \tag{23}$$

C2MRP

Consider an MRP representation $\vec{\sigma} \in \mathbb{R}^3$ and its corresponding DCM, $[C]$. To extract the MRP set from the DCM, we first compute the corresponding Euler Parameters (EPs) representation $\vec{\beta} = (\beta_0, \beta_1, \beta_2, \beta_3)$ using Sheppard's method.

Step 1: Find the largest value among

$$\beta_0^2 = \frac{1}{4}\left(1 + \text{tr}([C])\right), \quad \beta_1^2 = \frac{1}{4}\left(1 + 2C_{11} - \text{tr}([C])\right),$$

$$\beta_2^2 = \frac{1}{4}\left(1 + 2C_{22} - \text{tr}([C])\right), \quad \beta_3^2 = \frac{1}{4}\left(1 + 2C_{33} - \text{tr}([C])\right).$$

Step 2: Compute the remaining EPs using

$$\beta_0\beta_1 = \frac{C_{23} - C_{32}}{4}, \quad \beta_0\beta_2 = \frac{C_{31} - C_{13}}{4}, \quad \beta_0\beta_3 = \frac{C_{12} - C_{21}}{4},$$

$$\beta_1\beta_2 = \frac{C_{12} + C_{21}}{4}, \quad \beta_2\beta_3 = \frac{C_{23} + C_{32}}{4}, \quad \beta_3\beta_1 = \frac{C_{31} + C_{13}}{4}.$$

Lastly, the MRP attitude representation is obtained as

$$\sigma_i = \frac{\beta_i}{1 + \beta_0}, \quad i \in \{1, 2, 3\}.$$

G. Numerical Attitude Simulator

To integrate the attitude of the LMO satellite we design a numerical integrator based on a RK4 scheme. Since the inertial positions and velocities of both LMO and GMO satellites have already been calculated, the propagated attitude state is $\vec{X} \in \mathbb{R}^6$:

$$\vec{X}(t) = \begin{bmatrix} \vec{\sigma}_{\mathcal{B}/\mathcal{N}}(t) \\ {}^{\mathcal{B}}\vec{\omega}_{\mathcal{B}/\mathcal{N}}(t) \end{bmatrix}. \tag{24}$$

Since we have the initial state of the spacecraft $\vec{X}(t_0)$ in Section II.D, we only need differential equations to integrate the state. We express the differential equation for the angular velocity in the $\mathcal{B}$ frame because the inertia tensor is diagonal in that frame. Remember that the time derivative of an angular velocity is the same in all observers due to the Transport Theorem. We assume that the spacecraft is rigid and its dynamics obey:

$${}^{\mathcal{B}}[I]\,{}^{\mathcal{B}}\dot{\vec{\omega}}_{\mathcal{B}/\mathcal{N}} = -{}^{\mathcal{B}}[\tilde{\omega}_{\mathcal{B}/\mathcal{N}}]\,{}^{\mathcal{B}}[I]\,{}^{\mathcal{B}}\vec{\omega}_{\mathcal{B}/\mathcal{N}} + {}^{\mathcal{B}}\vec{u}, \tag{25}$$

where ${}^{\mathcal{B}}\vec{u}$ is an external control torque vector expressed in the $\mathcal{B}$ frame. Furthermore, the dynamics of the attitude MRP state $\vec{\sigma}_{\mathcal{B}/\mathcal{N}}$ is governed by the following differential equation:

$$\dot{\vec{\sigma}}_{\mathcal{B}/\mathcal{N}} = \frac{1}{4}[B(\vec{\sigma}_{\mathcal{B}/\mathcal{N}})]\,{}^{\mathcal{B}}\vec{\omega}_{\mathcal{B}/\mathcal{N}} = \frac{1}{4}\left[(1 - \vec{\sigma}_{\mathcal{B}/\mathcal{N}}^T\vec{\sigma}_{\mathcal{B}/\mathcal{N}})\,[I_{3\times3}] + 2[\tilde{\sigma}_{\mathcal{B}/\mathcal{N}}] + 2\,\vec{\sigma}_{\mathcal{B}/\mathcal{N}}\vec{\sigma}_{\mathcal{B}/\mathcal{N}}^T\right]^{\mathcal{B}}\vec{\omega}_{\mathcal{B}/\mathcal{N}} \tag{26}$$

To integrate the state $\vec{X}$, we implement a RK4 scheme with the following structure:

$$\dot{\vec{X}} = \vec{f}(t, \vec{X}, {}^{\mathcal{B}}\vec{u}) = \begin{bmatrix} \dot{\vec{\sigma}}_{\mathcal{B}/\mathcal{N}} \\ {}^{\mathcal{B}}\dot{\vec{\omega}}_{\mathcal{B}/\mathcal{N}} \end{bmatrix}, \tag{27}$$

where $\vec{f}$ defines the system dynamics, $\vec{X}$ is the state vector, and the time derivatives are determined by Eqs. 26 and 25. The pseu-docode for the RK4 scheme is detailed in Algorithm 1, including the shadow set switching logic for MRPs throughout the integration.

Algorithm 1 Pseudo-code RK4 Scheme

Inputs : $\vec{X}_0 = \begin{bmatrix} \vec{\sigma}_{\mathcal{B}/\mathcal{N}}(t_0) \\ {}^{\mathcal{B}}\vec{\omega}_{\mathcal{B}/\mathcal{N}}(t_0) \end{bmatrix}, \quad t_{\max}, \quad \Delta t, \quad t_n = 0.$

while $t_n < t_{\max}$ **do**

Determine control torque ${}^{\mathcal{B}}\vec{u}$ with Eq 28 and Table 3;

$\vec{X}_n = \vec{X}(t_n), \quad {}^{\mathcal{B}}\vec{u}_n = {}^{\mathcal{B}}\vec{u}(t_n);$

$\vec{k}_1 = \Delta t\, \vec{f}(\vec{X}_n, t_n, {}^{\mathcal{B}}\vec{u}_n);$

$\vec{k}_2 = \Delta t\, \vec{f}\left(\vec{X}_n + \frac{\vec{k}_1}{2},\, t_n + \frac{\Delta t}{2},\, {}^{\mathcal{B}}\vec{u}_n\right);$

$\vec{k}_3 = \Delta t\, \vec{f}\left(\vec{X}_n + \frac{\vec{k}_2}{2},\, t_n + \frac{\Delta t}{2},\, {}^{\mathcal{B}}\vec{u}_n\right);$

$\vec{k}_4 = \Delta t\, \vec{f}\left(\vec{X}_n + \vec{k}_3,\, t_n + \Delta t,\, {}^{\mathcal{B}}\vec{u}_n\right);$

$\vec{X}_{n+1} = \vec{X}_n + \frac{1}{6}\left(\vec{k}_1 + 2\vec{k}_2 + 2\vec{k}_3 + \vec{k}_4\right);$

if $\|\vec{\sigma}_{\mathcal{B}/\mathcal{N}}\| > 1$ **then**

map $\vec{\sigma}_{\mathcal{B}/\mathcal{N}}$ to shadow set: $\vec{\sigma}^{\,s}_{\mathcal{B}/\mathcal{N}} = -\frac{\vec{\sigma}_{\mathcal{B}/\mathcal{N}}}{\|\vec{\sigma}_{\mathcal{B}/\mathcal{N}}\|^2};$

end if

$t_{n+1} = t_n + \Delta t;$

save spacecraft states $\vec{X}_n$ and ${}^{\mathcal{B}}\vec{u}_n$;

end while

Outputs: $t_i, \vec{X}_i, {}^{\mathcal{B}}\vec{u}_i,\ i \in \{0, 1, 2, \cdots, N\}$

H. Attitude Control Law

In this work, we employ a simple proportional-derivative (PD) attitude control law of the form

$$ {}^{\mathcal{B}}\vec{u} = -K\vec{\sigma}_{\mathcal{B}/\mathcal{R}} - P\,{}^{\mathcal{B}}\vec{\omega}_{\mathcal{B}/\mathcal{R}}, \tag{28} $$

where $K \in \mathbb{R}^+$ is the feedback proportional gain, and $P \in \mathbb{R}^+$ is the feedback derivative gain. The goal of this control law is to drive the attitude tracking error $\vec{\sigma}_{\mathcal{B}/\mathcal{R}} = (\sigma_1, \sigma_2, \sigma_3)^T$ and the angular velocity tracking error ${}^{\mathcal{B}}\vec{\omega}_{\mathcal{B}/\mathcal{R}} = (\delta\omega_1, \delta\omega_2, \delta\omega_3)^T$ to zero, ensuring convergence of the body frame $\mathcal{B}$ to the desired reference frame $\mathcal{R}$. Throughout the mission, the reference frame $\mathcal{R} \in \{\mathcal{R}_s, \mathcal{R}_n, \mathcal{R}_c\}$ varies depending on the operational mode. To better understand the closed-loop behavior, we consider the nonlinear attitude dynamics introduced in Eqs. 25, 26, and 27. We linearize the dynamics about the equilibrium state $\vec{X} = \vec{0}$, corresponding to zero attitude and angular velocity tracking errors. The MRP kinematic equation simplifies to

$$ \dot{\vec{\sigma}}_{\mathcal{B}/\mathcal{R}} \approx \frac{1}{4}\,{}^{\mathcal{B}}\vec{\omega}_{\mathcal{B}/\mathcal{R}}. \tag{29} $$

Because the body frame $\mathcal{B}$ is aligned with the principal axes of inertia, the inertia matrix is diagonal, and the rotational dynamics decouple into three independent systems. As a result, the linearized closed-loop dynamics for each principal axis can be written in state-space formulation:

$$ \begin{bmatrix} \dot{\sigma}_i \\ \delta\dot{\omega}_i \end{bmatrix} = \begin{bmatrix} 0 & \frac{1}{4} \\ -\frac{K}{I_i} & -\frac{P}{I_i} \end{bmatrix} \begin{bmatrix} \sigma_i \\ \delta\omega_i \end{bmatrix}, \quad i \in \{1, 2, 3\}. \tag{30} $$

The roots of the corresponding characteristic equations can be expressed as:

$$ \lambda_i = -\frac{1}{2I_i}\left(P \pm \sqrt{-KI_i + P^2}\right), \quad i \in \{1, 2, 3\}. \tag{31} $$

Therefore, the feedback gains can be used to determine whether the system is under-damped (complex roots), critically damped (double real root), or over-damped (two unique real roots).

The system presented in Eq 30 is analogous to three uncoupled second-order spring–mass–damper systems, where the inertia I_i plays the role of mass, the proportional gain K acts as a spring stiffness, and the derivative gain P provides damping. To prove it, we take the second time derivative of σ_i:

$$\ddot{\sigma}_i + \frac{P}{I_i}\dot{\sigma}_i + \frac{K}{4I_i}\sigma_i = 0, \tag{32}$$

which is directly comparable to the form of a second-order spring-mass-damper system:

$$\ddot{x} + 2\zeta_i\omega_{n_i}\dot{x} + \omega_{n_i}^2 x = 0, \tag{33}$$

where ω_{n_i} the natural frequency and ζ_i is the damping ratio. With this comparison, we can calculate the following values for each principal axis:

$$\omega_{n_i} = \frac{\sqrt{K}}{2\sqrt{I_i}}, \quad \zeta_i = \frac{P}{\sqrt{KI_i}}, \quad \tau_i = \frac{2I_i}{P}, \tag{34}$$

where τ_i, known as the time decay constant, is the time required for the tracking error to decrease to approximately $1/e$ of its initial value. The damping ratio ζ_i determines whether the response is under-damped ($\zeta_i < 1$), critically damped ($\zeta_i = 1$), or overdamped ($\zeta_i > 1$).The under-damped regime displays a damped oscillatory motion, while the critically damped regime represents the fastest decay without oscillation. We are not interested in over-damped regimes because they are slower decays than the other two regimes.

The feedback gains are selected to achieve desired control performances. Specifically, the slowest decay time is chosen to be $\tau_{\max} = 120$ s. Since the largest inertia corresponds to axis $i = 1$, the derivative gain is selected as:

$$P = \frac{2I_{\max}}{\tau_{\max}} = \frac{2I_1}{120} = \frac{1}{6}\ \text{kg m}^2\text{s}^{-1}. \tag{35}$$

The corresponding decay time constants are:

$$\tau_1 = 120\ \text{s}, \quad \tau_2 = 60\ \text{s}, \quad \tau_3 = 90\ \text{s}. \tag{36}$$

In addition, one axis is chosen to be critically damped while the others remain under-damped. Enforcing $\zeta_{\max} = 1$ for the smallest inertia axis yields:

$$K = \frac{P^2}{I_{\min}} = \frac{P^2}{I_2} = \frac{1}{180}\ \text{kg m}^2\text{s}^{-2}. \tag{37}$$

The resulting damping ratios are:

$$\zeta_1 \approx 0.707, \quad \zeta_2 = 1, \quad \zeta_3 \approx 0.816, \tag{38}$$

indicating that axis 2 is critically damped, while axes 1 and 3 exhibit under-damped responses. Since $\zeta_1 = \zeta_{\min}$ and $\tau_1 = \tau_{\max}$, the spacecraft reorients most slowly about the axis with the largest inertia (axis 1), while faster convergence occurs along the other axes. The critically damped axis (axis 2) exhibits the fastest non-oscillatory response. Overall, we have guaranteed that the slowest decay time constant in our system is 120 s, while forcing one component to be critically damped and the other two under-damped.

I. Mission Pointing Modes

Based on the control law presented in Eq 28, three specific pointing control modes are defined:

- Sun Pointing Control (SPC): aligns the spacecraft with the Sun direction to maximize the use of solar panels.
- Nadir Pointing Control (NPC): aligns the spacecraft with the Mars nadir direction to gather science data.
- GMO Pointing Control (GPC): aligns the LMO nano-satellite with the GMO spacecraft for communication purposes.

Each mode uses the same PD control structure, differing only in the reference frame $\mathcal{R}$. The appropriate mode is selected based on the spacecraft's orbital position and the relative geometry between the LMO nano-satellite, Mars, the Sun, and the GMO satellite.

When the spacecraft is on the sunlit side of Mars, it operates in Sun-pointing mode to ensure that the solar panels are oriented toward the Sun. When the spacecraft is on the shaded side, it switches to either science or communication mode. Communication mode is selected when the angular separation between the LMO and GMO position vectors is less than 35°. Otherwise, the spacecraft operates in nadir-pointing mode to perform scientific observations. Table 3 summarizes the pointing modes. The angle γ between the LMO and GMO position vectors is defined as

$$\cos\gamma(t) = \frac{{}^{N}\vec{r}_{LMO}(t) \cdot {}^{N}\vec{r}_{GMO}(t)}{|{}^{N}\vec{r}_{LMO}(t)||{}^{N}\vec{r}_{GMO}(t)|}. \tag{39}$$

Table 3 Spacecraft Pointing Modes Summary

Mode	Reference Frame	Control Law	Condition
Sun	$\mathcal{R}_s$	${}^{\mathcal{B}}\vec{u}_s = -K\vec{\sigma}_{\mathcal{B}/\mathcal{R}_s} - P\,{}^{\mathcal{B}}\vec{\omega}_{\mathcal{B}/\mathcal{R}_s}$	${}^{N}\vec{r}_{LMO} \cdot \hat{n}_2 > 0$
Nadir	$\mathcal{R}_n$	${}^{\mathcal{B}}\vec{u}_n = -K\vec{\sigma}_{\mathcal{B}/\mathcal{R}_n} - P\,{}^{\mathcal{B}}\vec{\omega}_{\mathcal{B}/\mathcal{R}_n}$	${}^{N}\vec{r}_{LMO} \cdot \hat{n}_2 < 0,\ \gamma \geq 35°$
GMO	$\mathcal{R}_c$	${}^{\mathcal{B}}\vec{u}_c = -K\vec{\sigma}_{\mathcal{B}/\mathcal{R}_c} - P\,{}^{\mathcal{B}}\vec{\omega}_{\mathcal{B}/\mathcal{R}_c}$	${}^{N}\vec{r}_{LMO} \cdot \hat{n}_2 < 0,\ \gamma < 35°$

IV. Simulations & Discussion

Before combining all pointing modes into the full mission simulation, each control mode is developed and tested individually. For each case, a dedicated subsection analyzes a simplified scenario in which the spacecraft enters a given pointing mode at t_0 and remains in that mode for $t = 400$ s. After validating the performance of each control mode separately, the full mission scenario is simulated to evaluate the spacecraft's attitude pointing performance as it transitions between different control modes. Note that the initial attitude and angular velocity for all simulations are specified in Eqs. 2 and 3.

A. Sun Pointing Control Simulation

Figure 4 shows the time evolution of the spacecraft attitude and angular velocity during a SPC. The top plot exhibits a transition to the MRP shadow set at approximately $t \approx 170$ s, which ensures the MRP norm remains below unity without affecting the physical orientation. After this switch, the three components remain nearly constant, indicating convergence to a fixed inertial attitude. The bottom plot shows that all components of the angular velocity relative to the inertial frame decay to zero after approximately 75 s, confirming that the spacecraft reaches a non-rotating state since the rreference frame $\mathcal{R}_s$ is inertially fixed.

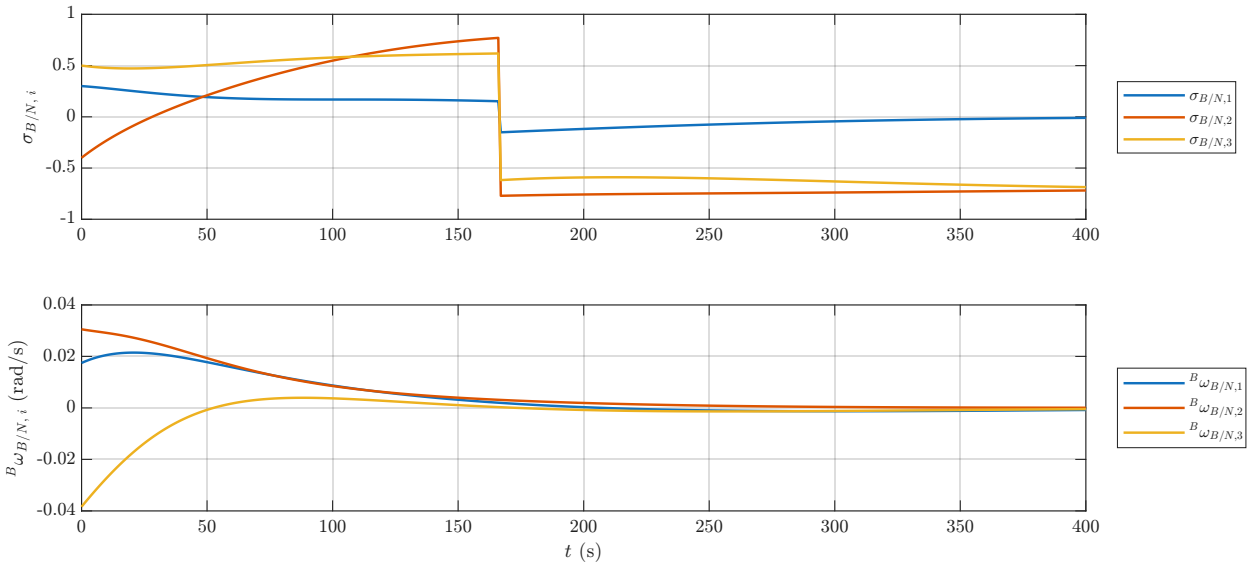

Fig. 4 Time evolution of spacecraft attitude and angular velocity during the SPC. Top: spacecraft attitude relative to the inertial frame, represented using MRPs. Bottom: spacecraft angular velocity relative to the inertial frame, expressed in the $\mathcal{B}$ frame.

Figure 5 presents the time evolution of the attitude tracking error, angular velocity tracking error, and control torque when the SPC is being applied. The attitude tracking error converges to zero, demonstrating asymptotic stability of the closed-loop system. The slowest convergence occurs along axis 1, consistent with the designed maximum time decay constant of $\tau_{\max} = \tau_1$. The damping behavior also matches the theoretical design: component 2 exhibits a critically damped response, while the components 1 and 3 display under-damped behavior. Similarly, the angular velocity tracking error decays to zero, confirming that rotational motion relative to the reference frame $\mathcal{R}_s$ is eliminated. The control torque is initially large but decreases smoothly to zero as the tracking errors vanish, consistent with the PD control law. Overall, the results confirm that the selected feedback gains yield the expected results for the SPC.

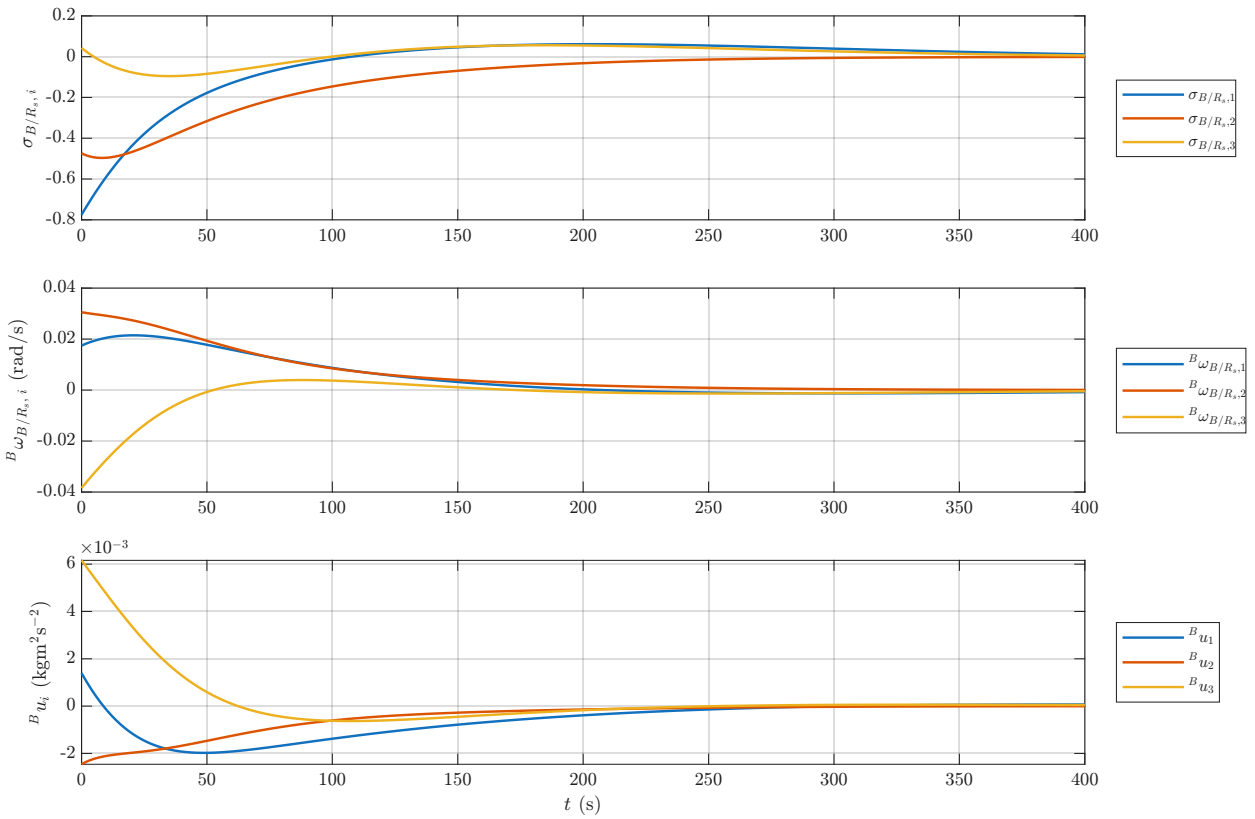

Fig. 5 Time evolution of tracking errors and control torque during the SPC. Top: attitude tracking error relative to frame $\mathcal{R}_s$, represented using MRPs. Middle: angular velocity tracking error relative to frame $\mathcal{R}_s$, expressed in the $\mathcal{B}$ frame. Bottom: control torque expressed in the $\mathcal{B}$ frame.

B. Nadir Pointing Control Simulation

The motion of the spacecraft during NPC is shown in Fig. 6. The attitude is represented using the original MRP set until approximately $t \approx 230$ s. After that, the second and third component of the MRP set remain constant while the first component slightly changes. Since component 1 has the slowest time decay constant, it takes longer for that component to be at the desired state.

Figure 7 displays the tracking errors and control torque during NPC. Both the attitude and angular velocity tracking errors asymptotically converge to zero. The top and middle subplots further illustrate the damping behavior: component 2 exhibits a critically damped response, while components 1 and 3 follow slightly under-damped regimes. Since components 1 and 3 are only lightly under-damped, their convergence rates remain close to that of component 2. The control torque decreases toward zero as the tracking errors vanish. Overall, the results confirm that the selected feedback gains provide stable and accurate tracking performance under NPC.

C. GMO Pointing Control Simulation

Unlike the previous control modes, Fig. 8 shows that the GPC does not cause the spacecraft attitude to switch to the MRP shadow set. Instead, the MRP components evolve continuously, reflecting the time-varying nature of the reference frame $\mathcal{R}_c$ associated with the GMO satellite. Figure 9 presents the time evolution of the tracking errors and control torque during GPC. Both the attitude and angular velocity tracking errors asymptotically converge to zero, confirming that the spacecraft successfully tracks the desired reference attitude and angular velocity. As the tracking errors vanish, the control torque also decreases toward zero, consistent with the PD control law.

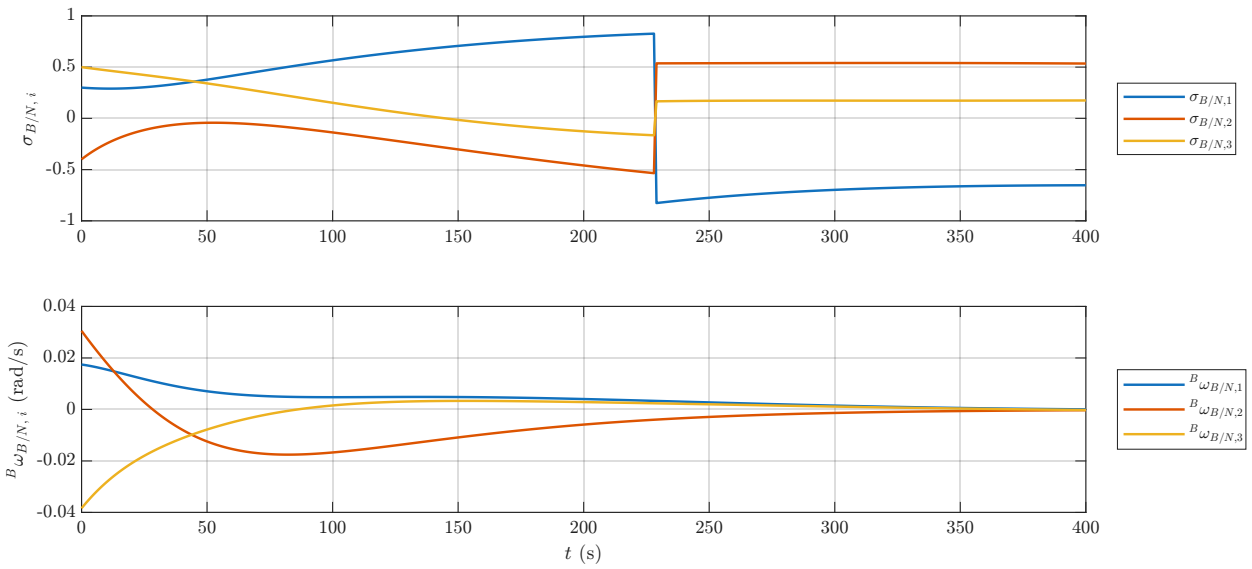

Fig. 6 Time evolution of spacecraft attitude and angular velocity during the NPC. Top: spacecraft attitude relative to the inertial frame, represented using MRPs. Bottom: spacecraft angular velocity relative to the inertial frame, expressed in the $\mathcal{B}$ frame.

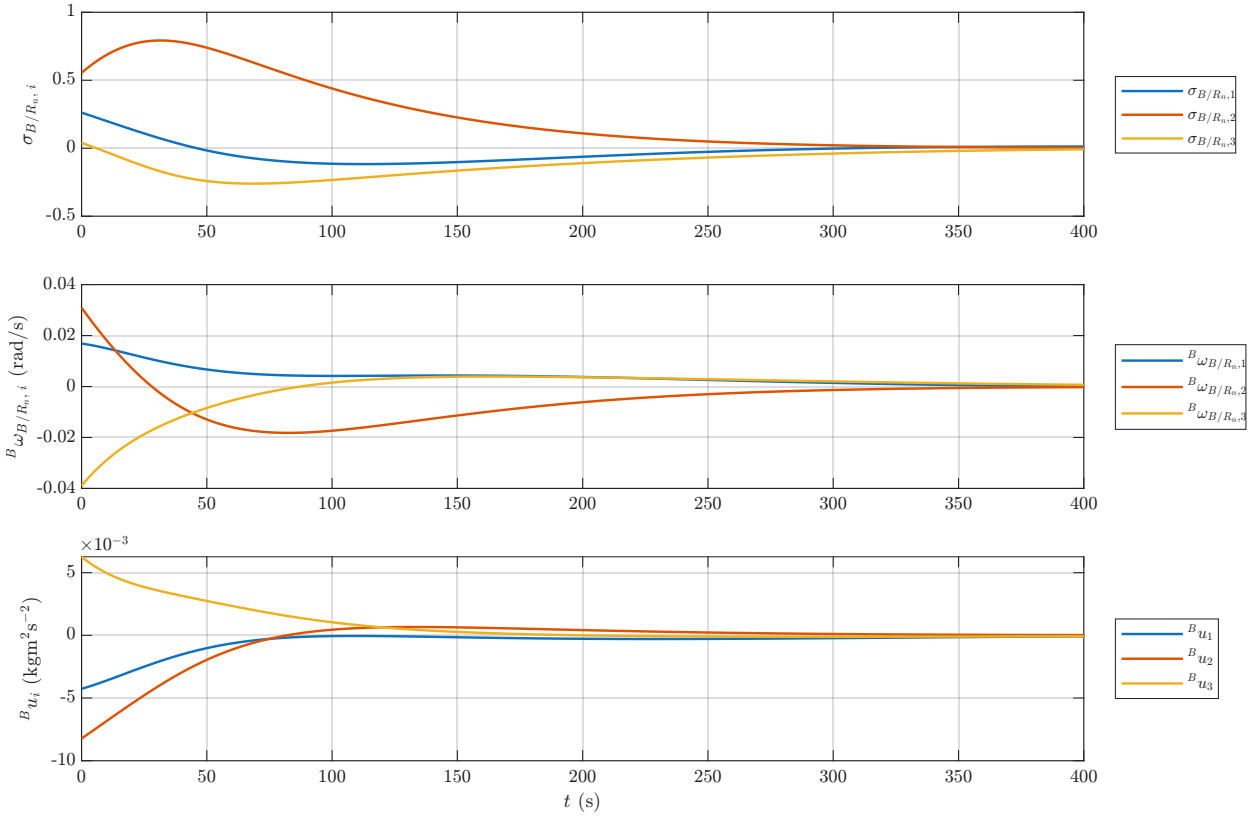

Fig. 7 Time evolution of tracking errors and control torque during the NPC. Top: attitude tracking error relative to frame $\mathcal{R}_n$, represented using MRPs. Middle: angular velocity tracking error relative to frame $\mathcal{R}_n$, expressed in the $\mathcal{B}$ frame. Bottom: control torque expressed in the $\mathcal{B}$ frame.

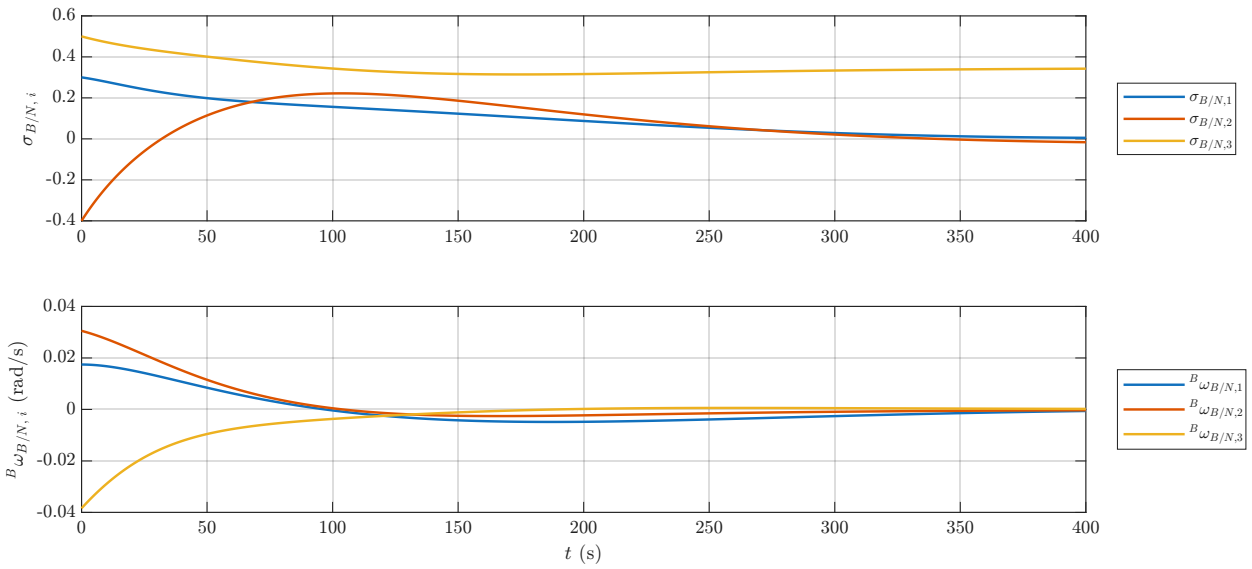

Fig. 8 Time evolution of spacecraft attitude and angular velocity during the GPC. Top: spacecraft attitude relative to the inertial frame, represented using MRPs. Bottom: spacecraft angular velocity relative to the inertial frame, expressed in the $\mathcal{B}$ frame.

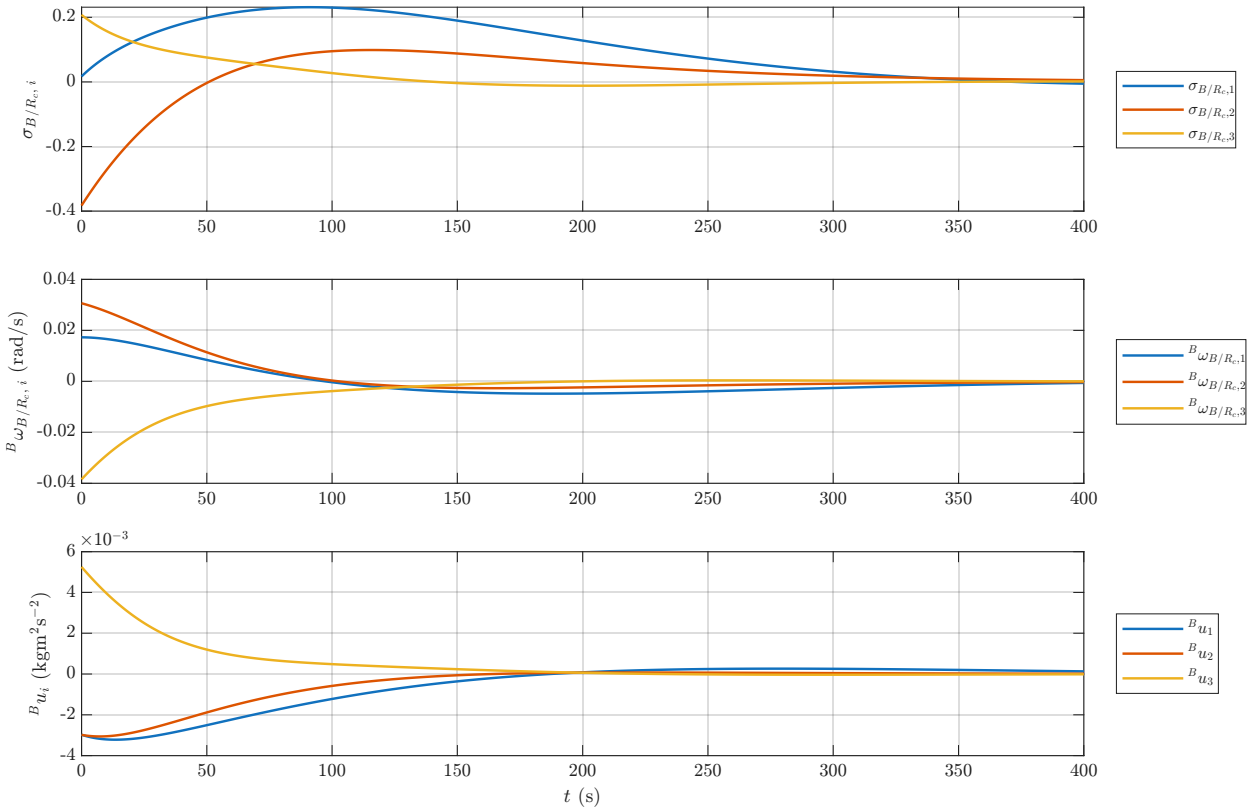

Fig. 9 Time evolution of tracking errors and control torque during the NPC. Top: attitude tracking error relative to frame $\mathcal{R}_c$, represented using MRPs. Middle: angular velocity tracking error relative to frame $\mathcal{R}_c$, expressed in the $\mathcal{B}$ frame. Bottom: control torque expressed in the $\mathcal{B}$ frame.

D. Mission Simulation

For the mission scenario simulation, the initial conditions in Eqs . 2 and 3. are propagated in time through 6500 s. Throughout the mission, the spacecraft autonomously switches modes according to the relative positions of both spacecrafts, as detailed in Table 3. Note that to calculate the inertial positions of the spacecrafts, we use the initial orbit conditions in Table 1 and the expression for $\theta(t)$ in Eq. 9. As a result, Fig. 10 displays how the pointing mode evolves throughout the whole mission. The Sun-pointing mode is enforced during the time interval $(0, 1918) \cup (5469, 6500)$ s, the Nadir-pointing mode is enforced during $(1918, 3057) \cup (4067, 5469)$ s, and the GMO-pointing mode is enforced only during $(3057, 4067)$ s. Therefore, we have successfully implemented mode-switching during the mission scenario simulation, where the spacecraft is able to complete all the necessary functions in order for the mission goals to be accomplished.

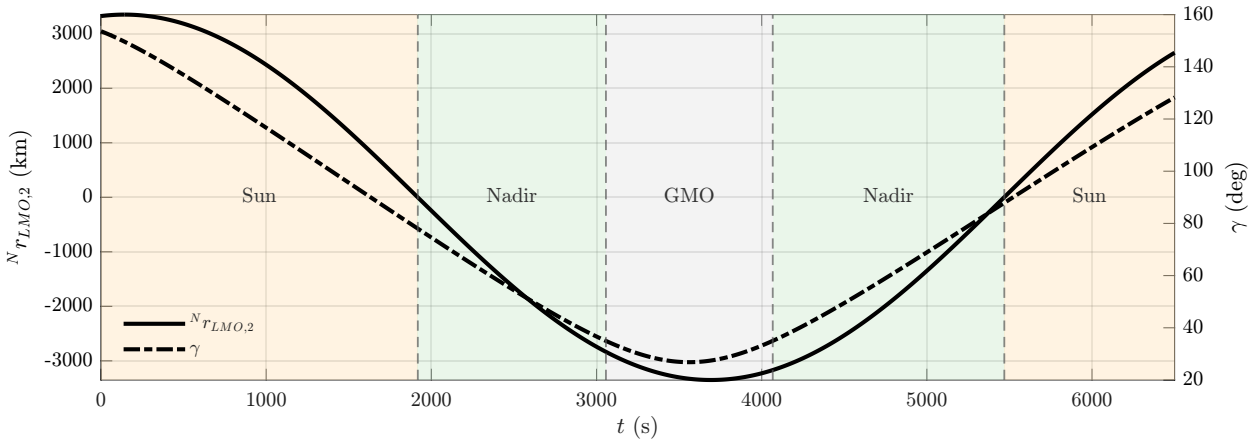

Fig. 10 Time evolution of the pointing modes during the proposed mission. The left y axis and the solid line represent the second component of the inertial position of the LMO satellite. The right y axis and the dashed line represent the angle γ between the LMO satellite and the GMO satellite. Each shaded region represents a different pointing mode, as the text label indicates. Different pointing modes are separated by thinner dashed lines.

Figure 11 depicts the evolution of the spacecraft's attitude and angular velocity with respect to the inertial frame throughout the whole mission. During the Sun-pointing time intervals (yellow regions), the attitude MRP state switches to the shadow set several times. This means that the spacecraft maintains a nearly constant attitude corresponding to a rotation near 180°, and small numerical variations lead the MRP set to repeatedly switch to the shadow set. During the Nadir-pointing time intervals (green regions), the MRP components continuously change to always point to the center of Mars. The same can be said for the GMO-pointing time intervals (gray regions), as the LMO spacecraft needs to continuously adapt to the new position of the mothercraft.

In addition, the angular velocity of the spacecraft with respect to the inertial frame remains small except during transitions between pointing modes, where brief increases are observed. This behavior arises because the reference frames themselves rotate slowly with respect to the inertial frame, so only a small angular velocity is required for the spacecraft to maintain alignment.

Figure 12 shows the time evolution of the attitude and angular velocity tracking errors relative to the active reference frame, as well as the applied control torque. The tracking errors remain close to zero throughout the mission, except during pointing mode changes, where brief deviations are observed due to the change in reference frame. These deviations are quickly corrected, indicating that the spacecraft is able to accurately reorient and maintain the desired pointing conditions. A similar behavior is observed in the angular velocity tracking error, which remains near zero during steady operation and only deviates during mode switches. As both the attitude and angular velocity tracking errors converge to zero, the control torque correspondingly decreases to zero, as expected from the PD control law. Overall, these results demonstrate that the spacecraft successfully achieves the desired pointing objectives throughout the mission.

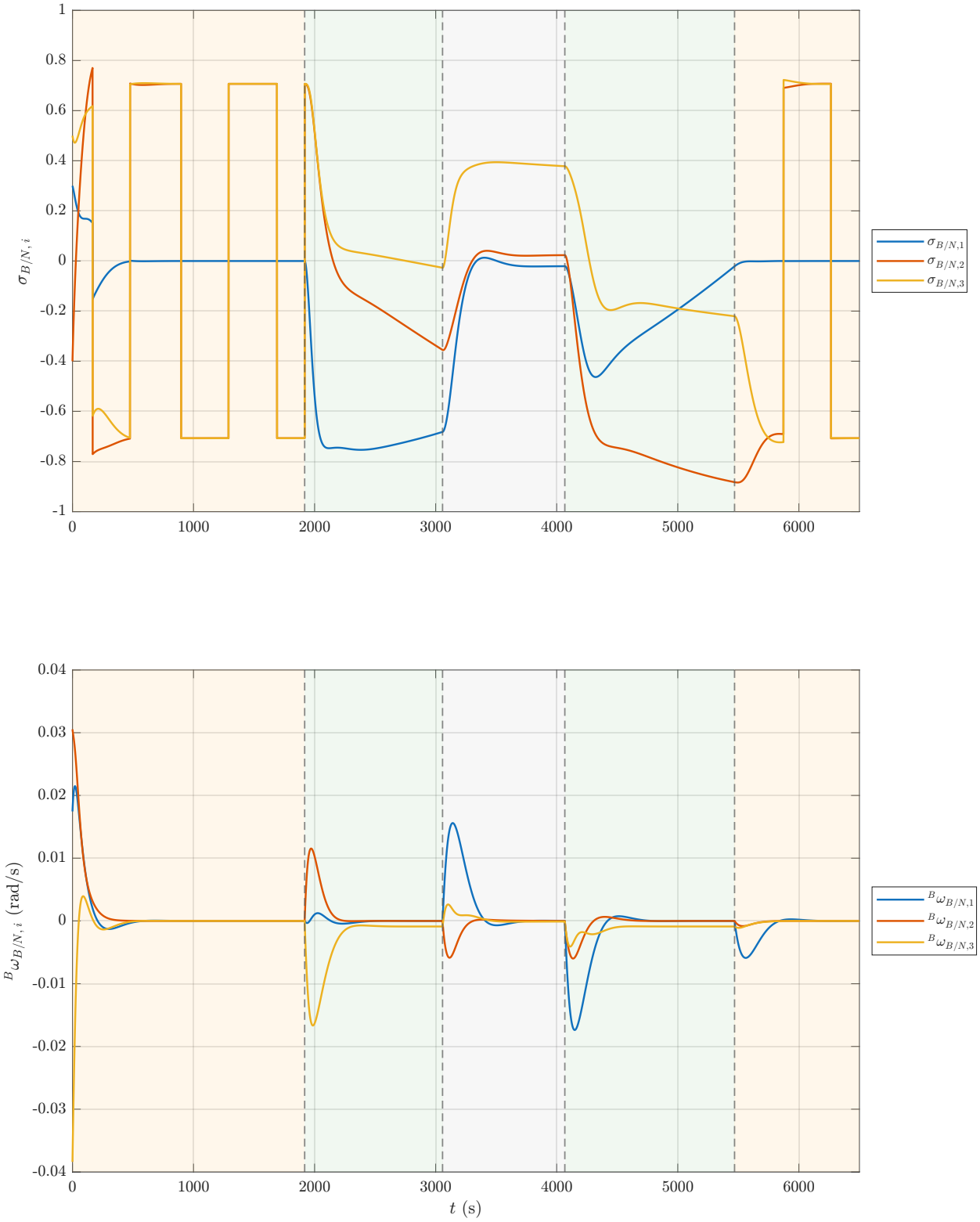

Fig. 11 Time evolution of spacecraft attitude and angular velocity during the proposed mission. Top: spacecraft attitude relative to the inertial frame, represented using MRPs. Bottom: spacecraft angular velocity relative to the inertial frame, expressed in the $\mathcal{B}$ frame. The shaded regions and the dashed lines are used to represent different pointing modes.

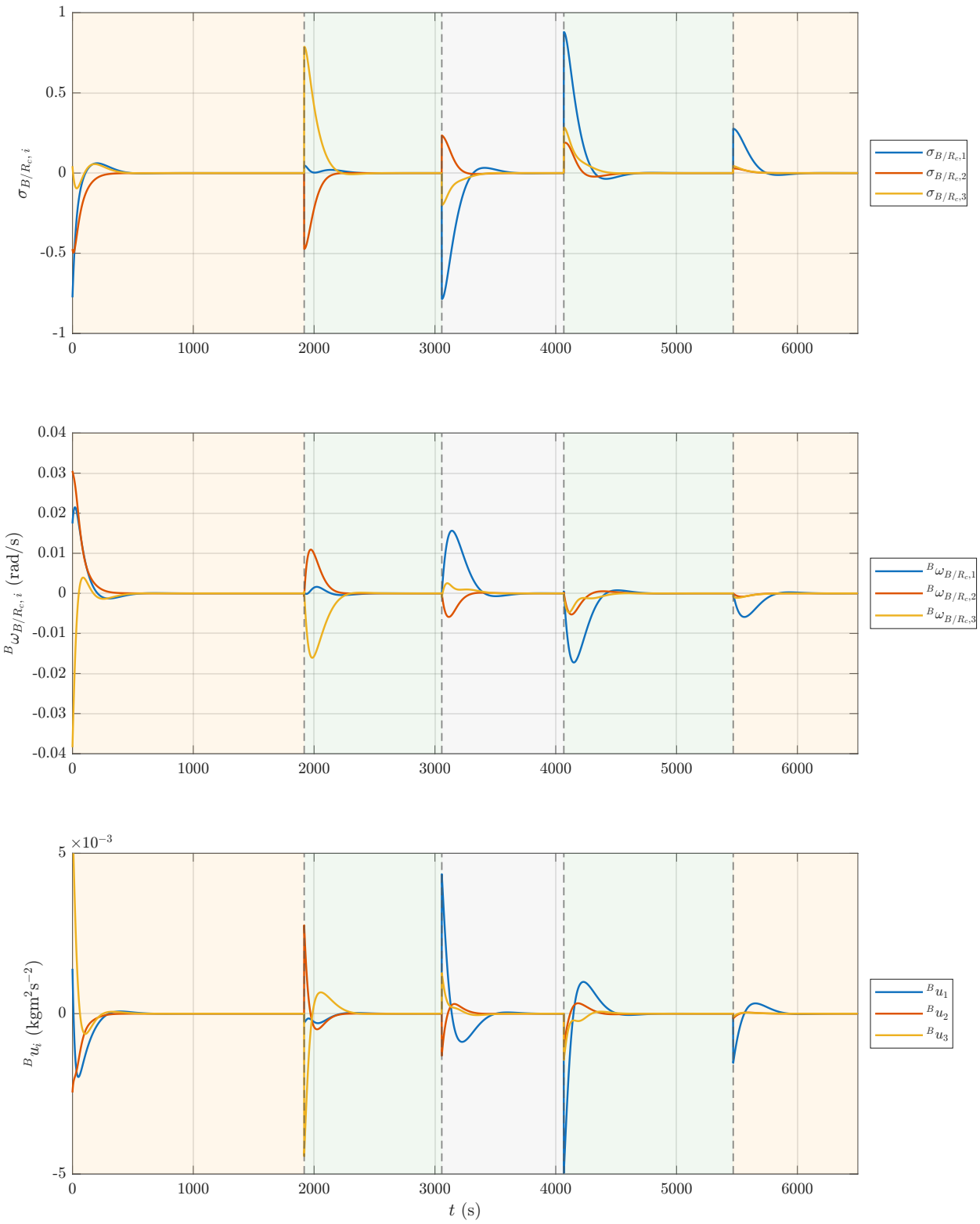

Fig. 12 Time evolution of tracking errors and control torque during the proposed mission. Top: attitude tracking error relative to the active reference frame, represented using MRPs. Second: angular velocity tracking error relative to the active reference frame, expressed in the $\mathcal{B}$ frame. Third: control torque expressed in the $\mathcal{B}$ frame. Bottom: mission geometry and operating modes as functions of the second component of the spacecraft inertial position and the angle γ between the two spacecrafts. The shaded regions and the dashed lines are used to represent different pointing modes.

V. Conclusion

This work presented the modeling, simulation, and control of a nano-satellite in Low Mars Orbit subject to multiple pointing requirements, including Sun-pointing, nadir-pointing, and communication with a mother spacecraft in Geosynchronous Mars Orbit. Each pointing mode was independently tested and shown to achieve accurate attitude tracking and asymptotic stability under a proportional-derivative control law. These modes were then integrated into an autonomous mode-switching framework based on mission geometry, enabling the spacecraft to transition between operational requirements throughout the mission. The full mission simulation demonstrates that the spacecraft consistently satisfies the desired pointing conditions, validating our autonomous mode-switching framework.

References

[1] Schaub, H., and Junkins, J. L., *Analytical Mechanics of Space Systems*, 4th ed., AIAA, Reston, VA, 2018.

Appendix

```

Global
Set up global parameters
% LMO satellite struct
LMO = struct();
LMO.r_Mars = 3396.19; % radius of Mars in km
LMO.h = 400; % altitude in km
LMO.r = LMO.r_Mars+ LMO.h; % orbit radius in km
LMO.theta_dot = 0.000884797; % orbit rate in rad/s
LMO.Omega = deg2rad(20); % argument of periapsis in rad
LMO.i = deg2rad(30); % inclination in rad
LMO.theta_t0 = deg2rad(60); % initial true anomaly in rad

% GMO satellite struct
GMO = struct();
GMO.r = 20424.2; % orbit radius in km
GMO.theta_dot = 0.0000709003; % orbit rate in rad/s
GMO.Omega = 0; % argument of periapsis in rad
GMO.i = 0; % inclination in rad
GMO.theta_t0 = deg2rad(250); % initial true anomaly in rad

% Struct with both LMO and GMO structs into a martian orbits struct
MOs = struct();
MOs.LMO = LMO;
Mos.GMO = GMO;

% Define spacecraft object
SC = struct();
SC.B_I_c = [10, 0, 0;
            0, 5, 0;
            0, 0, 7.5]; % inertia tensor in body frame B at center of mass c in kgm^2
SC.B_wBN_vec_t0 = [deg2rad(1); deg2rad(1.75); deg2rad(-2.2)]; % initial body angular
    velocity in rad/s
SC.sigmaBN_vec_t0 = [0.3 ; -0.4; 0.5]; % initial attitude
SC.X0_vec = [0.3 ; -0.4; 0.5;deg2rad(1); deg2rad(1.75); deg2rad(-2.2)]; % initial
    state

% Define control params
ctrl_PD = struct();
ctrl_PD.K = 1/180; % feedback gain in kgm^2/s^2
ctrl_PD.P = 1/6; % feedback gain in kgm^2/s
ctrl_PD.mode = 'Sun'; % it can be sun pointing mode, nadir pointing mode or
    communication pointing mode

% Generalized parameters
params_mission = struct();
params_mission.LMO = LMO;
params_mission.GMO = GMO;
params_mission.SC = SC;
params_mission.ctrl = ctrl_PD;

Task 1: Orbit Simulation
Functions
% function [N_r_vec, N_v_vec] = inertial_state(r, theta_dot, E313_vec)
% %=========================================================================
% %
% % Calculates inertial position vector and inertial velocity vector.
```

```
% % Circular orbit with constant radius and angular velocity is assumed.
% % Uses Dr. Schaub's rigid kinematics toolbox.
% % Refer to project pdf for N,O frame definitions and formulas used.
% %
% % Author: G. Montseny
% % Date: March 31, 2026
% %
% %
% % INPUT:              Description                                    Units
% %
% %  r    -            radius of circular orbit                         km
% %  theta_dot  -      angular velocity of circular orbit               rad/s
% %  E313_vec -        (3-1-3) Euler angles vector                      rad
% %
% % OUTPUT:
% %
% %  N_r_vec    -      inertial position (3x1)                          km
% %  N_v_vec    -      inertial velocity (3x1)                          km/s
% %=========================================================================
%
%     % Initialization & Inputs
%     E313_vec = E313_vec(:); % column shape
%
%     % Compute inertial vectors in O frame
%     O_r_vec = [r;0;0];
%     O_v_vec = [0;r*theta_dot;0];
%
%     % Compute DCM [NO]
%     ON = Euler3132C(E313_vec);
%     NO = ON';
%
%     % Compute inertial vectors in N frame
%     N_r_vec = NO*O_r_vec;
%     N_v_vec = NO*O_v_vec;
% end



function theta_t = theta(theta_t0, theta_dot, t)
%=========================================================================
%
% Author: G. Montseny
% Date: March 31, 2026
%
%
% INPUT:              Description                                  Units
%
%  theta_t0 -        true anomaly at initial time t0                rad
%  theta_dot  -      angular velocity of circular orbit             rad/s
%  t    -            time                                           s
%
% OUTPUT:
%
%  theta_t    -      true anomaly at final time t                   rad/s
%=========================================================================

    % Formula
    theta_t = theta_t0 + theta_dot*t;

```

```
end

function [N_r_vec, N_v_vec] = inertial_state(params_orb, t)
%=========================================================================
%
% Calculates inertial position vector and inertial velocity vector.
% Circular orbit with constant radius and angular velocity is assumed.
% Uses Dr. Schaub's rigid kinematics toolbox.
% Refer to project pdf for N,O frame definitions and formulas used.
%
% Author: G. Montseny
% Date: March 31, 2026
%
%
% INPUT:               Description                                    Units
%
%  params_orb    -   orbtial parameters  (LMO/GMO)                     -
%  t          -       time                                           s
%
% OUTPUT:
%
%  N_r_vec     -     inertial position (3x1)                          km
%  N_v_vec     -     inertial velocity (3x1)                          km/s
%=========================================================================

    % Initialization & Inputs
    r = params_orb.r;
    theta_dot = params_orb.theta_dot;
    theta_t0 = params_orb.theta_t0;

    % Compute theta_t and Euler vector
    theta_t = theta(theta_t0, theta_dot, t);
    E313_vec = [params_orb.Omega; params_orb.i; theta_t];

    % Compute inertial vectors in O frame
    O_r_vec = [r;0;0];
    O_v_vec = [0;r*theta_dot;0];

    % Compute DCM [NO]
    ON = Euler3132C(E313_vec);
    NO = ON';

    % Compute inertial vectors in N frame
    N_r_vec = NO*O_r_vec;
    N_v_vec = NO*O_v_vec;
end
Code
LMO
% theta_t = theta(params.LMO.theta_t0, params.LMO.theta_dot, 450);
% E313_vec = [params.LMO.Omega; params.LMO.i; theta_t];
% [N_rLMO_vec, N_vLMO_vec] = inertial_state(params.LMO.r, params.LMO.theta_dot,
    E313_vec);
[N_rLMO_vec, N_vLMO_vec] = inertial_state(params.LMO, 450)
GMO
% theta_t = theta(params.GMO.theta_t0, params.GMO.theta_dot, 1150);
% E313_vec = [params.GMO.Omega; params.GMO.i; theta_t];
% [N_rGMO_vec, N_vGMO_vec] = inertial_state(params.GMO.r, params.GMO.theta_dot,
    E313_vec);
[N_rGMO_vec, N_vGMO_vec] = inertial_state(params.GMO, 1150)
```

```
Submission formatting
file = fopen(fullfile(outputFolder,'Task1_1_v2.txt'), 'w');
fprintf(file, '%.5f␣%.5f␣%.5f', N_rLMO_vec);
fclose(file);

file = fopen(fullfile(outputFolder,'Task1_2_v2.txt'), 'w');
fprintf(file, '%.5f␣%.5f␣%.5f', N_vLMO_vec);
fclose(file);

file = fopen(fullfile(outputFolder,'Task1_3_v2.txt'), 'w');
fprintf(file, '%.5f␣%.5f␣%.5f', N_rGMO_vec);
fclose(file);

file = fopen(fullfile(outputFolder,'Task1_4_v2.txt'), 'w');
fprintf(file, '%.5f␣%.5f␣%.5f', N_vGMO_vec);
fclose(file);

Task 2: Orbit Frame Orientation
Functions
function HN = N2H(params_orb, t)
%=========================================================================
%
% Calculates [HN] DCM for time t.
% For LMO or GMO.
%
% Author: G. Montseny
% Date: March 31, 2026
%
%
% INPUT:               Description                                  Units
%
%  params_orb    -   orbtial parameters  (LMO/GMO)                    -
%  t          -       time                                           s
%
% OUTPUT:
%
%  HN         -       DCM from N frame to H frame (3x3)              -
%=========================================================================

    % Calculate true anomaly
    theta_t = theta(params_orb.theta_t0, params_orb.theta_dot, t);

    % Extract Euler angles column vector
    E313_vec = [params_orb.Omega; params_orb.i; theta_t];

    % Compute DCM [HN]
    HN = Euler3132C(E313_vec);
end


Code
HN_300 = N2H(params.LMO, 300)
Submission Formatting
file = fopen(fullfile(outputFolder,'Task2.txt'), 'w');
fprintf(file, '%.5f␣%.5f␣%.5f␣%.5f␣%.5f␣%.5f␣%.5f␣%.5f␣%.5f', HN_300');
fclose(file);

Task 3: Sun-Pointing Reference Frame Orientation
Functions
```

```
function RsN = N2Rs()
%=========================================================================
%
% Calculates RsN DCM, where Rs is solar-facing reference frame.
%
% Author: G. Montseny
% Date: March 31, 2026
%
%
% INPUT:               Description                                  Units
%
%  t          -       time                                           s
%
% OUTPUT:
%
%  RsN        -       DCM from N frame to Rs frame (3x3)             -
%=========================================================================

RsN = [-1,0,0;
    0,0,1;
    0,1,0];
end


Code
Submission Formatting
file = fopen(fullfile(outputFolder,'Task3_1.txt'), 'w');
fprintf(file, '%.5f␣%.5f␣%.5f␣%.5f␣%.5f␣%.5f␣%.5f␣%.5f␣%.5f', N2Rs');
fclose(file);
file = fopen(fullfile(outputFolder,'Task3_2.txt'), 'w');
fprintf(file, '%.5f␣%.5f␣%.5f', [0;0;0]);
fclose(file);

Task 4: Nadir-Pointing Reference Frame Orientation
Functions
function RnN = N2Rn(params_orb, t)
%=========================================================================
%
% Calculates [RnN] DCM for time t.
% Spacecraft sensor points towards the center of Mars/nadir direction.
% For LMO or GMO.
%
% Author: G. Montseny
% Date: March 31, 2026
%
%
% INPUT:               Description                                  Units
%
%  params_orb   -   orbtial parameters  (LMO/GMO)                    -
%  t          -       time                                           s
%
% OUTPUT:
%
%  RnN        -       DCM from N frame to Rn frame (3x3)             -
%=========================================================================

    % Compute RnH
    RnH = [-1,0,0;
        0,1,0;
```

```
        0,0,-1];

    % Compute HN
    HN = N2H(params_orb, t);

    % Cmpute RnN
    RnN = RnH*HN;
end

function N_w_RnN_vec = inertial_w_RnN(params_orb, t)
%=========================================================================
%
% Calculates N_w_RnN_vec for time t.
% It's the angular velocity of frame Rn wrt to inertial frame N,
% expressed (coordinatized) in N frame coordinates.
% For LMO or GMO.
%
% Author: G. Montseny
% Date: March 31, 2026
%
%
% INPUT:               Description                                  Units
%
%  params_orb    -   orbtial parameters  (LMO/GMO)                    -
%  t          -       time                                           s
%
% OUTPUT:
%
%  N_w_RnN_vec   -    angular velocity of Rn frame wrt to N frame      rad/s
%=========================================================================

    % Compute NH
    HN = N2H(params_orb, t);
    NH = HN';

    % Final formula
    N_w_RnN_vec = NH*[0;0;params_orb.theta_dot];
end


Code
RnN_330 = N2Rn(LMO, 330);
N_w_RnN_vec_330 = inertial_w_RnN(LMO, 330);
Submission Formatting
file = fopen(fullfile(outputFolder,'Task4_1.txt'), 'w');
fprintf(file, '%.5f␣%.5f␣%.5f␣%.5f␣%.5f␣%.5f␣%.5f␣%.5f␣%.5f', RnN_330');
fclose(file);

file = fopen(fullfile(outputFolder,'Task4_2.txt'), 'w');
fprintf(file, '%.9f␣%.9f␣%.9f', N_w_RnN_vec_330);
fclose(file);

Task 5: GMO - Pointing Reference Frame Orientation
Functions
function RcN = N2Rc(params_glob, t)
%=========================================================================
%
% Calculates [RcN] DCM for time t.
% Communications mode.
```

```
%
% Author: G. Montseny
% Date: March 31, 2026
%
%
% INPUT:                Description                                     Units
%
%  params_glob    -       global parameters of the problem               -
%  t          -       time                                              s
%
% OUTPUT:
%
%  RcN         -       DCM from N frame to Rc frame (3x3)               -
%=========================================================================

    % Access parameters for both orbits
    params_LMO = params_glob.LMO;
    params_GMO = params_glob.GMO;

    % Compute orbit DCMs for LMO and GMO
    HN_LMO = N2H(params_LMO, t); NH_LMO = HN_LMO';
    HN_GMO = N2H(params_GMO, t); NH_GMO = HN_GMO';

    % Compute N_r1_hat
    N_rLMO_vec = NH_LMO*[params_LMO.r;0;0];
    N_rGMO_vec = NH_GMO*[params_GMO.r;0;0];

    N_Delta_r_vec = N_rGMO_vec - N_rLMO_vec;

    N_r1_hat = - N_Delta_r_vec/norm(N_Delta_r_vec);

    % Compute N_r2_hat
    N_r2_vec = cross(N_Delta_r_vec,[0;0;1]);
    N_r2_hat = N_r2_vec/norm(N_r2_vec);

    % Compute N_r3_hat
    N_r3_hat = cross(N_r1_hat,N_r2_hat);

    % Compute DCM
    RcN = [N_r1_hat';N_r2_hat';N_r3_hat'];

end

function N_w_RcN_vec = inertial_w_RcN(params_glob, t)
%=========================================================================
%
% Calculates N_w_RcN_vec for time t.
% It's the angular velocity of frame Rc wrt to inertial frame N,
% expressed (coordinatized) in N frame coordinates.
% Numerical differences are used to compute the derivative of RcN
%
% Author: G. Montseny
% Date: March 31, 2026
%
%
% INPUT:                Description                                     Units
%
%  params_glob    -       global parameters of the problem               -
%  t          -            time                                         s
```

```
%
% OUTPUT:
%
%  N_w_RnN_vec  -    angular velocity of Rn frame wrt to N frame      rad/s
%==========================================================================

    % Compute RcN
    RcN_t = N2Rc(params_glob, t);

    % Compute Rc_wRcN_tilde through numerical differences
    dt = 1e-5;
    RcN_tpdt = N2Rc(params_glob, t+dt);

    if t > dt

        % Central difference
        RcN_tmdt = N2Rc(params_glob, t-dt);
        RcN_dot = (RcN_tpdt - RcN_tmdt)/(2*dt);

    else
        % Forward difference
        RcN_dot = (RcN_tpdt - RcN_t)/dt;
    end

    Rc_wRcN_tilde = - RcN_dot*RcN_t';

    % Change to inertial frame
    N_wRcN_tilde = RcN_t'*Rc_wRcN_tilde*RcN_t;

    % Extract vector from skew operator
    N_w_RcN_vec = [N_wRcN_tilde(3,2); N_wRcN_tilde(1,3); N_wRcN_tilde(2,1)];
end
Code
RcN_330 = N2Rc(params, 330);
N_w_RcN_vec_330 = inertial_w_RcN(params, 330);
Submission Formatting
file = fopen(fullfile(outputFolder,'Task5_1.txt'), 'w');
fprintf(file, '%.5f␣%.5f␣%.5f␣%.5f␣%.5f␣%.5f␣%.5f␣%.5f', RcN_330');
fclose(file);

file = fopen(fullfile(outputFolder,'Task5_2.txt'), 'w');
fprintf(file, '%.8f␣%.8f␣%.8f', N_w_RcN_vec_330);
fclose(file);

Task 6: Attitude Error Evaluation
Functions
function [sigma_BR_vec, B_w_BR_vec] = track_err(sigma_BN_vec, B_w_BN_vec, RN,
    N_w_RN_vec)
%==========================================================================
%
% Calculates associated tracking errors at time t.
% Tracking errors between body frame B and reference frame R.
% N frame is the inertial frame.
% All inputs and outputs are given at time t.
%
% Author: G. Montseny
% Date: April 1, 2026
%
%
```

```
% INPUT:               Description                                    Units
%
% sigma_BN_vec         MRP vector B frame wrt N frame (3x1)           -
% B_w_BN_vec           angular speed of frame B wrt to frame N,       rad/s
%                      coordinatized in B frame (3x1)
% NR                   DCM from N frame to B frame (3x3)              -
% N_w_RN_vec           angular speed of frame R wrt to frame N,       rad/s
%                      coordinatized in N frame (3x1)
%
%
% OUTPUT:
%
% sigma_BR_vec         MRP vector B frame wrt R frame (3x1)           -
% B_w_BR_vec           angular speed of frame B wrt to frame R,       rad/s
%                      coordinatized in B frame (3x1)
%=========================================================================

    % Initialization
    sigma_BN_vec = sigma_BN_vec(:);
    B_w_BN_vec = B_w_BN_vec(:);
    N_w_RN_vec = N_w_RN_vec(:);

    % Calculate [BN](t)
    BN = MRP2C(sigma_BN_vec);

    % Calculate B_w_BR_vec(t)
    B_w_BR_vec = B_w_BN_vec-BN*N_w_RN_vec;

    % Calculate sigma_BR_vec(t)
    BR = BN*RN';
    sigma_BR_vec = C2MRP(BR);

end

Code
% Body Frame B Inputs
sigma_BN_vec_t0 = [0.3; -0.4; 0.5];
B_w_BN_vec_t0 = [deg2rad(1); deg2rad(1.75); deg2rad(-2.2)]; % rad/s

% Sun-Pointing frame Rs
RsN_t0 = N2Rs();
N_w_RsN_vec_t0 = [0; 0; 0];
[sigma_BRs_vec_t0, B_w_BRs_vec_t0] = track_err(sigma_BN_vec_t0, B_w_BN_vec_t0, RsN_t0,
     N_w_RsN_vec_t0);

% Nadir-Pointing frame Rn
RnN_t0 = N2Rn(params.LMO, 0);
N_w_RnN_vec_t0 = inertial_w_RnN(params.LMO, 0);
[sigma_BRn_vec_t0, B_w_BRn_vec_t0] = track_err(sigma_BN_vec_t0, B_w_BN_vec_t0, RnN_t0,
     N_w_RnN_vec_t0);

% GMO-Pointing frame Rc
RcN_t0 = N2Rc(params, 0);
N_w_RcN_vec_t0 = inertial_w_RcN(params, 0);
[sigma_BRc_vec_t0, B_w_BRc_vec_t0] = track_err(sigma_BN_vec_t0, B_w_BN_vec_t0, RcN_t0,
     N_w_RcN_vec_t0);

Submission Formatting
file = fopen(fullfile(outputFolder,'Task6_1.txt'), 'w');
```

```
fprintf(file, '%.5f %.5f %.5f', sigma_BRs_vec_t0);
fclose(file);

file = fopen(fullfile(outputFolder,'Task6_2.txt'), 'w');
fprintf(file, '%.5f %.5f %.5f', B_w_BRs_vec_t0);
fclose(file);

file = fopen(fullfile(outputFolder,'Task6_3.txt'), 'w');
fprintf(file, '%.5f %.5f %.5f', sigma_BRn_vec_t0);
fclose(file);

file = fopen(fullfile(outputFolder,'Task6_4.txt'), 'w');
fprintf(file, '%.5f %.5f %.5f', B_w_BRn_vec_t0);
fclose(file);

file = fopen(fullfile(outputFolder,'Task6_5.txt'), 'w');
fprintf(file, '%.5f %.5f %.5f', sigma_BRc_vec_t0);
fclose(file);

file = fopen(fullfile(outputFolder,'Task6_6.txt'), 'w');
fprintf(file, '%.5f %.5f %.5f', B_w_BRc_vec_t0);
fclose(file);

Task 7: Numerical Attitude Simulator
Functions
function [X_vec_hist, u_vec_hist, t_hist] = RK4(f, u_fun, X0_vec, t_span, dt, params)
%==========================================================================
%
% Integrates X_dot = f(t, X, u, params) using a fixed-step RK4 scheme
%
% The structure follows the class algorithm:
%   1) evaluate control
%   2) perform one RK4 step
%   3) apply MRP shadow-set switch if needed
%   4) save state and control histories
%
% Author: G. Montseny
% Date: April 6, 2026
%
%
% INPUT:              Description                                    Units
%
%  f          -   dynamics function handle f(t, X_vec, u_vec, params_SC)  -
%  u_fun      -   control function handle u_fun(t, X_vec, params_ctrl)      -
%  X0_vec     -   initial state vector                                    -
%  t_span     -   integration time span                                   s
%  dt         -   fixed integration time step                             s
%  params     -   structure with all parameters                           -
%
% OUTPUT:
%
%  X_vec_hist -   state history, one column per time step                 -
%  u_vec_hist -   control history, one column per time step               -
%  t_hist     -   time history                                            s
%
%==========================================================================

    %--------------------------------------------------------------------
    %--------------------------------------------------------------------
```

```
    %                       INITIALIZATION
    %-------------------------------------------------------------------------
    %-------------------------------------------------------------------------

    % Make sure initial state is a column vector
    X0_vec = X0_vec(:);

    % Initial time and final time
    t0 = t_span(1);
    tf = t_span(2);

    % Number of time steps
    N = floor((tf - t0)/dt) + 1;

    % Time history
    t_hist = t0 + (0:N-1)*dt;

    % Add final time if needed
    if abs(t_hist(end) - tf) > 1e-12
        t_hist = [t_hist, t_max];
        N = length(t_hist);
    end

    % State size
    n_X = length(X0_vec);

    % Evaluate control once to get control size
    u0_vec = u_fun(t0, X0_vec, params);
    u0_vec = u0_vec(:);
    n_u = length(u0_vec);

    % Initialize histories
    X_vec_hist = zeros(n_X, N);
    u_vec_hist = zeros(n_u, N);

    % Store initial values
    X_vec_hist(:,1) = X0_vec;
    u_vec_hist(:,1) = u0_vec;

    %-------------------------------------------------------------------------
    %-------------------------------------------------------------------------
    %                       TIME-STEPPING LOOP
    %-------------------------------------------------------------------------
    %-------------------------------------------------------------------------

    for n = 1:N-1

        % Current time and state
        t_n = t_hist(n);
        X_n_vec = X_vec_hist(:,n);

        %-----------------------------------------------------------------
        % Evaluate control at current step
        % This matches the class logic where the control is updated
        % once per integration step and then held fixed during RK4
        %-----------------------------------------------------------------
        u_n_vec = u_fun(t_n, X_n_vec, params);
        u_n_vec = u_n_vec(:);
        % INCLUDE MORE STUFF IN CLASS
```

```
639
640         %-------------------------------------------------------------
641         % RK4 integration step
642         %-------------------------------------------------------------
643         h = dt;
644         k1_vec = h * f(t_n,           X_n_vec,             u_n_vec, params);
645         k2_vec = h * f(t_n + h/2,     X_n_vec + k1_vec/2,  u_n_vec, params);
646         k3_vec = h * f(t_n + h/2,     X_n_vec + k2_vec/2,  u_n_vec, params);
647         k4_vec = h * f(t_n + h,       X_n_vec + k3_vec,    u_n_vec, params);
648
649         X_np1_vec = X_n_vec + (1/6) * (k1_vec + 2*k2_vec + 2*k3_vec + k4_vec);
650
651         %-------------------------------------------------------------
652         % MRP shadow-set switching
653         % Assume the first 3 states are the MRP vector sigma_B/N
654         %-------------------------------------------------------------
655         sigma_BN_vec = X_np1_vec(1:3);
656
657         if norm(sigma_BN_vec) > 1
658             X_np1_vec(1:3) = -sigma_BN_vec / norm(sigma_BN_vec)^2;
659         end
660
661         %-------------------------------------------------------------
662         % Save next state and current control
663         %-------------------------------------------------------------
664         X_vec_hist(:,n+1) = X_np1_vec;
665         u_vec_hist(:,n) = u_n_vec;
666
667     end
668
669     % Store final control value
670     u_vec_hist(:,N) = u_fun(t_hist(N), X_vec_hist(:,N), params);
671
672 end
673
674 function Xdot_vec = f_T7(t, X_vec, u_vec, params_mission)
675
676     params_SC = params_mission.SC;
677     % CHANGE TO VALID FOR ALL INERTIA TENSORS, NOT ONLY DIAGONAL/PRINCIPAL
678     % Column X_vec and u_vec
679     X_vec = X_vec(:);
680     u_vec = u_vec(:);
681
682     % Initialize Xdot_vec
683     Xdot_vec = zeros(size(X_vec));
684
685     % Extract values
686     B_wBN_vec = X_vec(4:6);
687     sigmaBN_vec = X_vec(1:3);
688
689     w_1 = B_wBN_vec(1); w_2 = B_wBN_vec(2); w_3 = B_wBN_vec(3);
690     u_1 = u_vec(1); u_2 = u_vec(2); u_3 = u_vec(3);
691
692     % Extract moment of inertia values (assuming B is a principal axes
693     % frame)
694     B_I_c = params_SC.B_I_c;
695     I_1 = B_I_c(1,1); I_2 = B_I_c(2,2); I_3 = B_I_c(3,3);
696
697     % Differential equations for sigma_i
```

```
    Xdot_vec(1:3) = dMRP(sigmaBN_vec,B_wBN_vec);

    % Differential equations for w_i
    Xdot_vec(4) = (-(I_3-I_2)*w_2*w_3 + u_1)/I_1;
    Xdot_vec(5) = (-(I_1-I_3)*w_1*w_3 + u_2)/I_2;
    Xdot_vec(6) = (-(I_2-I_1)*w_1*w_2 + u_3)/I_3;

end



function H_vec = angularMomentum(SC, B_wBN_vec)
    H_vec = SC.B_I_c*B_wBN_vec;
end


Code
u_fun = @(t, X_vec, SC) [0;0;0];
[X_vec_hist, u_vec_hist, t_hist] = RK4(@f_T7, u_fun, SC.X0_vec, [0,500], 1, SC);
t_hist(end)
B_wBN_vec_500 = X_vec_hist(4:6,end)
B_H_vec = SC.B_I_c*B_wBN_vec_500
T_rot_500 = 0.5*B_wBN_vec_500'*SC.B_I_c*B_wBN_vec_500
sigmaBN_vec_500 = X_vec_hist(1:3,end)
BN_500 = MRP2C(sigmaBN_vec_500);
N_H_vec = BN_500'*B_H_vec

u_fun = @(t, X_vec, SC) [0.01;-0.01;0.02];
[X_vec_hist, u_vec_hist, t_hist] = RK4(@f_T7, u_fun, SC.X0_vec, [0,100], 1, SC);
sigmaBN_vec_100 = X_vec_hist(1:3,end)

Submission Formatting
file = fopen(fullfile(outputFolder,'Task7_1.txt'), 'w');
fprintf(file, '%.5f␣%.5f␣%.5f', B_H_vec);
fclose(file);

file = fopen(fullfile(outputFolder,'Task7_2.txt'), 'w');
fprintf(file, '%.5f', T_rot_500);
fclose(file);

file = fopen(fullfile(outputFolder,'Task7_3.txt'), 'w');
fprintf(file, '%.5f␣%.5f␣%.5f', sigmaBN_vec_500);
fclose(file);

file = fopen(fullfile(outputFolder,'Task7_4.txt'), 'w');
fprintf(file, '%.5f␣%.5f␣%.5f', N_H_vec);
fclose(file);

file = fopen(fullfile(outputFolder,'Task7_5.txt'), 'w');
fprintf(file, '%.5f␣%.5f␣%.5f', sigmaBN_vec_100);
fclose(file);

Task 8: Sun Pointing Control
Functions
function B_u_vec = attitude_pointing_control(t, X_vec, params_mission)

    % Extract parameters
    params_orb = params_mission.LMO;
    params_ctrl = params_mission.ctrl;
```

```
757
758     % Extract state
759     sigma_BN_vec = X_vec(1:3);
760     B_w_BN_vec = X_vec(4:6);
761
762     % Determine reference DCM and angular velocity
763     switch params_ctrl.mode
764         case 'Sun'
765             RN = N2Rs();
766             N_w_RN_vec = [0; 0; 0];
767         case 'Nadir'
768             RN = N2Rn(params_orb, t);
769             N_w_RN_vec = inertial_w_RnN(params_orb,t);
770         case 'GMO'
771             RN = N2Rc(params_mission, t);
772             N_w_RN_vec = inertial_w_RcN(params_mission, t);
773         otherwise
774             error('Given␣pointing␣mode␣does␣not␣exist')
775     end
776
777     % Calculate tracking errors
778     [sigma_BR_vec, B_w_BR_vec] = track_err(sigma_BN_vec, B_w_BN_vec, RN, N_w_RN_vec);
779
780     % Extract control gains
781     P = params_ctrl.P;
782     K = params_ctrl.K;
783
784     % Simple PD Control Law
785     B_u_vec = -K*sigma_BR_vec-P*B_w_BR_vec;
786 end
787 Code
788 params_mission.ctrl.mode = 'Sun';
789 u_Sun = @(t,X_vec, params_mission) attitude_pointing_control(t,X_vec, params_mission);
790 [X_vec_hist, u_vec_hist, t_hist] = RK4(@f_T7, u_Sun, SC.X0_vec, [0,400], 1,
        params_mission);
791 sigma_BN_vec_15 = X_vec_hist(1:3, 16);
792 sigma_BN_vec_100 = X_vec_hist(1:3, 101);
793 sigma_BN_vec_200 = X_vec_hist(1:3, 201);
794 sigma_BN_vec_400 = X_vec_hist(1:3, 401);
795 Submission
796 file = fopen(fullfile(outputFolder,'Task8_1.txt'), 'w');
797 fprintf(file, '%.7f␣%.7f', [ctrl_PD.P; ctrl_PD.K]);
798 fclose(file);
799
800 file = fopen(fullfile(outputFolder,'Task8_2.txt'), 'w');
801 fprintf(file, '%.7f␣%.7f␣%.7f', sigma_BN_vec_15);
802 fclose(file);
803
804 file = fopen(fullfile(outputFolder,'Task8_3.txt'), 'w');
805 fprintf(file, '%.7f␣%.7f␣%.7f', sigma_BN_vec_100);
806 fclose(file);
807
808 file = fopen(fullfile(outputFolder,'Task8_4.txt'), 'w');
809 fprintf(file, '%.7f␣%.7f␣%.7f', sigma_BN_vec_200);
810 fclose(file);
811
812 file = fopen(fullfile(outputFolder,'Task8_5.txt'), 'w');
813 fprintf(file, '%.7f␣%.7f␣%.7f', sigma_BN_vec_400);
814 fclose(file);
```

```
SC Motion Plots
figure('Position', [100, 100, 900, 400]); % [left, bottom, width, height]
tiledlayout(2,1)

nexttile
plot(t_hist, X_vec_hist(1,:),   'LineWidth',1.2); hold on;
plot(t_hist, X_vec_hist(2,:) ,'LineWidth',1.2); hold on;
plot(t_hist, X_vec_hist(3,:),  'LineWidth',1.2); hold on;
%plot(t_hist, vecnorm(X_vec_hist(1:3,:))); hold on;
grid on;
ylabel('$\sigma_{{B/N,\;i}}$'); %lxlabel('$t$ (s)');
legend('$\sigma_{{B/N,1}}$', '$\sigma_{{B/N,2}}$', '$\sigma_{{B/N,3}}$', 'Location', '
    eastoutside');

nexttile
plot(t_hist, X_vec_hist(4,:),   'LineWidth',1.2); hold on;
plot(t_hist, X_vec_hist(5,:),   'LineWidth',1.2); hold on;
plot(t_hist, X_vec_hist(6,:),   'LineWidth',1.2); hold on;
grid on;
ylabel('$^B\omega_{{B/N,\;i}}$␣(rad/s)'); xlabel('$t$␣(s)');
legend('$^B\omega_{{B/N,1}}$', '$^B\omega_{{B/N,2}}$', '$^B\omega_{{B/N,3}}$', '
    Location', 'eastoutside');
exportgraphics(gcf, 'Sun_1.pdf', 'ContentType','vector');
Tracking Performance & Control
% CALCULATE
sigma_BN_vec_hist = X_vec_hist(1:3, :);
N = size(sigma_BN_vec_hist,2);
sigma_BR_vec_hist = zeros(3,N);
RsN = N2Rs();
for k = 1:N
    % Extract sigma_BN at time k
    sigma_BN_vec = sigma_BN_vec_hist(:,k);

    % Convert to DCM
    BN = MRP2C(sigma_BN_vec);

    % Compose attitude
    BR = BN * RsN';

    % Convert back to MRP
    sigma_BR_vec = C2MRP(BR);

    % Store
    sigma_BR_vec_hist(:,k) = sigma_BR_vec;
end

B_w_BR_vec_hist = X_vec_hist(4:6, :);


% Plot
figure('Position', [100, 100, 900, 600]); % [left, bottom, width, height]
tiledlayout(3,1)

nexttile
plot(t_hist, sigma_BR_vec_hist(1,:),   'LineWidth',1.2); hold on;
plot(t_hist, sigma_BR_vec_hist(2,:),   'LineWidth',1.2); hold on;
plot(t_hist, sigma_BR_vec_hist(3,:),   'LineWidth',1.2); hold on;
%plot(t_hist, vecnorm(X_vec_hist(1:3,:))); hold on;
grid on;
```

```
ylabel('$\sigma_{{B/R_s,\;i}}$'); %xlabel('$t$ (s)');
legend('$\sigma_{{B/R_s,1}}$', '$\sigma_{{B/R_s,2}}$', '$\sigma_{{B/R_s,3}}$', '
    Location', 'eastoutside');

nexttile
plot(t_hist, B_w_BR_vec_hist(1,:),   'LineWidth',1.2); hold on;
plot(t_hist, B_w_BR_vec_hist(2,:),   'LineWidth',1.2); hold on;
plot(t_hist, B_w_BR_vec_hist(3,:),   'LineWidth',1.2); hold on;
grid on;
ylabel('$^B\omega_{{B/R_s,\;i}}$␣(rad/s)'); %xlabel('$t$ (s)');
legend('$^B\omega_{{B/R_s,1}}$', '$^B\omega_{{B/R_s,2}}$', '$^B\omega_{{B/R_s,3}}$', '
    Location', 'eastoutside');


nexttile
plot(t_hist, u_vec_hist(1,:),   'LineWidth',1.2); hold on;
plot(t_hist, u_vec_hist(2,:),   'LineWidth',1.2); hold on;
plot(t_hist, u_vec_hist(3,:),   'LineWidth',1.2); hold on;
grid on;
ylabel('$^Bu_i$␣(kgm$^2$s$^{-2}$)'); xlabel('$t$␣(s)');
legend('$^Bu_1$','$^Bu_2$', '$^Bu_3$', 'Location', 'eastoutside');
exportgraphics(gcf, 'Sun_2.pdf', 'ContentType','vector');

Task 9: Nadir Pointing Control
Code
params_mission.ctrl.mode = 'Nadir';
u_Nadir = @(t,X_vec, params_mission) attitude_pointing_control(t,X_vec, params_mission
    );
[X_vec_hist, u_vec_hist, t_hist] = RK4(@f_T7, u_Nadir, SC.X0_vec, [0,400], 1,
    params_mission);
sigma_BN_vec_15 = X_vec_hist(1:3, 16);
sigma_BN_vec_100 = X_vec_hist(1:3, 101);
sigma_BN_vec_200 = X_vec_hist(1:3, 201);
sigma_BN_vec_400 = X_vec_hist(1:3, 401);
Submission
file = fopen(fullfile(outputFolder,'Task9_1.txt'), 'w');
fprintf(file, '%.7f␣%.7f␣%.7f', sigma_BN_vec_15);
fclose(file);

file = fopen(fullfile(outputFolder,'Task9_2.txt'), 'w');
fprintf(file, '%.7f␣%.7f␣%.7f', sigma_BN_vec_100);
fclose(file);

file = fopen(fullfile(outputFolder,'Task9_3.txt'), 'w');
fprintf(file, '%.7f␣%.7f␣%.7f', sigma_BN_vec_200);
fclose(file);

file = fopen(fullfile(outputFolder,'Task9_4.txt'), 'w');
fprintf(file, '%.7f␣%.7f␣%.7f', sigma_BN_vec_400);
fclose(file);
SC Motion Plots
figure('Position', [100, 100, 900, 400]); % [left, bottom, width, height]
tiledlayout(2,1)

nexttile
plot(t_hist, X_vec_hist(1,:),   'LineWidth',1.2); hold on;
plot(t_hist, X_vec_hist(2,:) ,'LineWidth',1.2); hold on;
plot(t_hist, X_vec_hist(3,:),  'LineWidth',1.2); hold on;
%plot(t_hist, vecnorm(X_vec_hist(1:3,:))); hold on;
```

```
grid on;
ylabel('$\sigma_{{B/N,\;i}}$'); %lxlabel('$t$ (s)');
legend('$\sigma_{{B/N,1}}$', '$\sigma_{{B/N,2}}$', '$\sigma_{{B/N,3}}$', 'Location', '
    eastoutside');

nexttile
plot(t_hist, X_vec_hist(4,:),   'LineWidth',1.2); hold on;
plot(t_hist, X_vec_hist(5,:),   'LineWidth',1.2); hold on;
plot(t_hist, X_vec_hist(6,:),   'LineWidth',1.2); hold on;
grid on;
ylabel('$^B\omega_{{B/N,\;i}}$␣(rad/s)'); xlabel('$t$␣(s)');
legend('$^B\omega_{{B/N,1}}$', '$^B\omega_{{B/N,2}}$', '$^B\omega_{{B/N,3}}$', '
    Location', 'eastoutside');
exportgraphics(gcf, 'Nadir_1.pdf', 'ContentType','vector');
Tracking Performance & Control
% CALCULATE
sigma_BN_vec_hist = X_vec_hist(1:3, :);
B_w_BN_vec_hist = X_vec_hist(4:6, :);
N = size(sigma_BN_vec_hist,2);
sigma_BR_vec_hist = zeros(3,N);
B_w_BR_vec_hist = zeros(3,N);


for k = 1:N
    % Extract sigma_BN and w_BN at time k
    sigma_BN_vec = sigma_BN_vec_hist(:,k);
    B_w_BN_vec = B_w_BN_vec_hist(:,k);

    % Extract time k
    t_k = t_hist(k);

    % Extract RnN
    RnN = N2Rn(LMO, t_k);

    % Extract w_RnN
    N_w_RnN_vec = inertial_w_RnN(LMO, t_k);

    % Calculate track errors
    [sigma_BR_vec, B_w_BR_vec] = track_err(sigma_BN_vec, B_w_BN_vec, RnN, N_w_RnN_vec)
        ;

    % Store
    sigma_BR_vec_hist(:,k) = sigma_BR_vec;
    B_w_BR_vec_hist(:,k) = B_w_BR_vec;


end



% Plot
figure('Position', [100, 100, 900, 600]); % [left, bottom, width, height]
tiledlayout(3,1)

nexttile
plot(t_hist, sigma_BR_vec_hist(1,:),   'LineWidth',1.2); hold on;
plot(t_hist, sigma_BR_vec_hist(2,:),   'LineWidth',1.2); hold on;
plot(t_hist, sigma_BR_vec_hist(3,:),   'LineWidth',1.2); hold on;
%plot(t_hist, vecnorm(X_vec_hist(1:3,:))); hold on;
```

```
grid on;
ylabel('$\sigma_{{B/R_n,\;i}}$'); %xlabel('$t$ (s)');
legend('$\sigma_{{B/R_n,1}}$', '$\sigma_{{B/R_n,2}}$', '$\sigma_{{B/R_n,3}}$', '
    Location', 'eastoutside');

nexttile
plot(t_hist, B_w_BR_vec_hist(1,:),   'LineWidth',1.2); hold on;
plot(t_hist, B_w_BR_vec_hist(2,:),   'LineWidth',1.2); hold on;
plot(t_hist, B_w_BR_vec_hist(3,:),   'LineWidth',1.2); hold on;
grid on;
ylabel('$^B\omega_{{B/R_n,\;i}}$␣(rad/s)'); %xlabel('$t$ (s)');
legend('$^B\omega_{{B/R_n,1}}$', '$^B\omega_{{B/R_n,2}}$', '$^B\omega_{{B/R_n,3}}$', '
    Location', 'eastoutside');


nexttile
plot(t_hist, u_vec_hist(1,:),   'LineWidth',1.2); hold on;
plot(t_hist, u_vec_hist(2,:),   'LineWidth',1.2); hold on;
plot(t_hist, u_vec_hist(3,:),   'LineWidth',1.2); hold on;
grid on;
ylabel('$^Bu_i$␣(kgm$^2$s$^{-2}$)'); xlabel('$t$␣(s)');
legend('$^Bu_1$','$^Bu_2$', '$^Bu_3$', 'Location', 'eastoutside');
exportgraphics(gcf, 'Nadir_2.pdf', 'ContentType','vector');

Task 10: GMO Pointing Control
Code
params_mission.ctrl.mode = 'GMO';
u_GMO = @(t,X_vec, params_mission) attitude_pointing_control(t,X_vec, params_mission);
[X_vec_hist, u_vec_hist, t_hist] = RK4(@f_T7, u_GMO, SC.X0_vec, [0,400], 1,
    params_mission);
sigma_BN_vec_15 = X_vec_hist(1:3, 16);
sigma_BN_vec_100 = X_vec_hist(1:3, 101);
sigma_BN_vec_200 = X_vec_hist(1:3, 201);
sigma_BN_vec_400 = X_vec_hist(1:3, 401);
Submission
file = fopen(fullfile(outputFolder,'Task10_1.txt'), 'w');
fprintf(file, '%.7f␣%.7f␣%.7f', sigma_BN_vec_15);
fclose(file);

file = fopen(fullfile(outputFolder,'Task10_2.txt'), 'w');
fprintf(file, '%.7f␣%.7f␣%.7f', sigma_BN_vec_100);
fclose(file);

file = fopen(fullfile(outputFolder,'Task10_3.txt'), 'w');
fprintf(file, '%.7f␣%.7f␣%.7f', sigma_BN_vec_200);
fclose(file);

file = fopen(fullfile(outputFolder,'Task10_4.txt'), 'w');
fprintf(file, '%.7f␣%.7f␣%.7f', sigma_BN_vec_400);
fclose(file);
SC Motion Plots
figure('Position', [100, 100, 900, 400]); % [left, bottom, width, height]
tiledlayout(2,1)

nexttile
plot(t_hist, X_vec_hist(1,:),   'LineWidth',1.2); hold on;
plot(t_hist, X_vec_hist(2,:) ,'LineWidth',1.2); hold on;
plot(t_hist, X_vec_hist(3,:),  'LineWidth',1.2); hold on;
%plot(t_hist, vecnorm(X_vec_hist(1:3,:))); hold on;
```

```
grid on;
ylabel('$\sigma_{{B/N,\;i}}$'); %lxlabel('$t$ (s)');
legend('$\sigma_{{B/N,1}}$', '$\sigma_{{B/N,2}}$', '$\sigma_{{B/N,3}}$', 'Location', '
    eastoutside');

nexttile
plot(t_hist, X_vec_hist(4,:),   'LineWidth',1.2); hold on;
plot(t_hist, X_vec_hist(5,:),   'LineWidth',1.2); hold on;
plot(t_hist, X_vec_hist(6,:),   'LineWidth',1.2); hold on;
grid on;
ylabel('$^B\omega_{{B/N,\;i}}$␣(rad/s)'); xlabel('$t$␣(s)');
legend('$^B\omega_{{B/N,1}}$', '$^B\omega_{{B/N,2}}$', '$^B\omega_{{B/N,3}}$', '
    Location', 'eastoutside');
exportgraphics(gcf, 'GMO_1.pdf', 'ContentType','vector');
Tracking Performance & Control
% CALCULATE
sigma_BN_vec_hist = X_vec_hist(1:3, :);
B_w_BN_vec_hist = X_vec_hist(4:6, :);
N = size(sigma_BN_vec_hist,2);
sigma_BR_vec_hist = zeros(3,N);
B_w_BR_vec_hist = zeros(3,N);


for k = 1:N
    % Extract sigma_BN and w_BN at time k
    sigma_BN_vec = sigma_BN_vec_hist(:,k);
    B_w_BN_vec = B_w_BN_vec_hist(:,k);

    % Extract time k
    t_k = t_hist(k);

    % Extract RcN
    RcN = N2Rc(params_mission, t_k);

    % Extract w_RcN
    N_w_RcN_vec = inertial_w_RcN(params_mission, t_k);

    % Calculate track errors
    [sigma_BR_vec, B_w_BR_vec] = track_err(sigma_BN_vec, B_w_BN_vec, RcN, N_w_RcN_vec)
        ;

    % Store
    sigma_BR_vec_hist(:,k) = sigma_BR_vec;
    B_w_BR_vec_hist(:,k) = B_w_BR_vec;

end



% Plot
figure('Position', [100, 100, 900, 600]); % [left, bottom, width, height]
tiledlayout(3,1)

nexttile
plot(t_hist, sigma_BR_vec_hist(1,:),   'LineWidth',1.2); hold on;
plot(t_hist, sigma_BR_vec_hist(2,:),   'LineWidth',1.2); hold on;
plot(t_hist, sigma_BR_vec_hist(3,:),   'LineWidth',1.2); hold on;
%plot(t_hist, vecnorm(X_vec_hist(1:3,:))); hold on;
grid on;
```

```
ylabel('$\sigma_{{B/R_c,\;i}}$'); %xlabel('$t$ (s)');
legend('$\sigma_{{B/R_c,1}}$', '$\sigma_{{B/R_c,2}}$', '$\sigma_{{B/R_c,3}}$', '
    Location', 'eastoutside');

nexttile
plot(t_hist, B_w_BR_vec_hist(1,:),   'LineWidth',1.2); hold on;
plot(t_hist, B_w_BR_vec_hist(2,:),   'LineWidth',1.2); hold on;
plot(t_hist, B_w_BR_vec_hist(3,:),   'LineWidth',1.2); hold on;
grid on;
ylabel('$^B\omega_{{B/R_c,\;i}}$␣(rad/s)'); %xlabel('$t$ (s)');
legend('$^B\omega_{{B/R_c,1}}$', '$^B\omega_{{B/R_c,2}}$', '$^B\omega_{{B/R_c,3}}$', '
    Location', 'eastoutside');


nexttile
plot(t_hist, u_vec_hist(1,:),   'LineWidth',1.2); hold on;
plot(t_hist, u_vec_hist(2,:),   'LineWidth',1.2); hold on;
plot(t_hist, u_vec_hist(3,:),   'LineWidth',1.2); hold on;
grid on;
ylabel('$^Bu_i$␣(kgm$^2$s$^{-2}$)'); xlabel('$t$␣(s)');
legend('$^Bu_1$','$^Bu_2$', '$^Bu_3$', 'Location', 'eastoutside');
exportgraphics(gcf, 'GMO_2.pdf', 'ContentType','vector');

Task 11: Mission Scenario Simulation
Functions
function [X_vec_hist, u_vec_hist, t_hist] = integrate_mission(f, u_fun, X0_vec, t_span
    , dt, params)
%=========================================================================
%
% Integrates X_dot = f(t, X, u, params) using a fixed-step RK4 scheme
%
% The structure follows the class algorithm:
%   1) evaluate control
%   2) perform one RK4 step
%   3) apply MRP shadow-set switch if needed
%   4) save state and control histories
% SOLVE PARAMS INPUT: SHOULD ID EVEN BE ONE?
%
% Author: G. Montseny
% Date: April 6, 2026
%
%
% INPUT:               Description                                    Units
%
%  f          -   dynamics function handle f(t, X_vec, u_vec, params_SC)  -
%  u_fun      -   control function handle u_fun(t, X_vec, params_ctrl)      -
%  X0_vec     -   initial state vector                                    -
%  t_span     -   integration time span                                   s
%  dt         -   fixed integration time step                             s
%  params     -   structure with all parameters                           -
%
% OUTPUT:
%
%  X_vec_hist -   state history, one column per time step                 -
%  u_vec_hist -   control history, one column per time step               -
%  t_hist     -   time history                                            s
%
%=========================================================================

```

```
    %-----------------------------------------------------------------------
    %-----------------------------------------------------------------------
    %                         INITIALIZATION
    %-----------------------------------------------------------------------
    %-----------------------------------------------------------------------

    % Make sure initial state is a column vector
    X0_vec = X0_vec(:);

    % Initial time and final time
    t0 = t_span(1);
    tf = t_span(2);

    % Number of time steps
    N = floor((tf - t0)/dt) + 1;

    % Time history
    t_hist = t0 + (0:N-1)*dt;

    % Add final time if needed
    if abs(t_hist(end) - tf) > 1e-12
        t_hist = [t_hist, t_max];
        N = length(t_hist);
    end

    % State size
    n_X = length(X0_vec);

    % Evaluate control once to get control size
    u0_vec = u_fun(t0, X0_vec, params);
    u0_vec = u0_vec(:);
    n_u = length(u0_vec);

    % Initialize histories
    X_vec_hist = zeros(n_X, N);
    u_vec_hist = zeros(n_u, N);

    % Store initial values
    X_vec_hist(:,1) = X0_vec;
    u_vec_hist(:,1) = u0_vec;

    %-----------------------------------------------------------------------
    %-----------------------------------------------------------------------
    %                         TIME-STEPPING LOOP
    %-----------------------------------------------------------------------
    %-----------------------------------------------------------------------

    for n = 1:N-1

        % Current time and state
        t_n = t_hist(n);
        X_n_vec = X_vec_hist(:,n);

        %---------------------------------------------------------------
        % Evaluate control at current step
        % This matches the class logic where the control is updated
        % once per integration step and then held fixed during RK4
        %---------------------------------------------------------------
```

```
        % Extract LMO and GMO posiitons
        [N_rLMO_vec, ~] = inertial_state(params.LMO, t_n); r_LMO = norm(N_rLMO_vec);
        [N_rGMO_vec, ~] = inertial_state(params.GMO, t_n); r_GMO = norm(N_rGMO_vec);


        % Conditions
        if dot(N_rLMO_vec, [0;1;0]) > 0 % SUN
            params.ctrl.mode = 'Sun';
        else
            gamma = acosd(N_rLMO_vec'*N_rGMO_vec/(r_LMO*r_GMO));

            if gamma >= 35 % NADIR
                params.ctrl.mode = 'Nadir';

            else % GMO
                params.ctrl.mode = 'GMO';

            end

        end

        % Control vector
        u_n_vec = u_fun(t_n, X_n_vec, params);
        u_n_vec = u_n_vec(:);

        %---------------------------------------------------------------
        % RK4 integration step
        %---------------------------------------------------------------
        h = dt;
        k1_vec = h * f(t_n,           X_n_vec,           u_n_vec, params);
        k2_vec = h * f(t_n + h/2,     X_n_vec + k1_vec/2,  u_n_vec, params);
        k3_vec = h * f(t_n + h/2,     X_n_vec + k2_vec/2,  u_n_vec, params);
        k4_vec = h * f(t_n + h,       X_n_vec + k3_vec,    u_n_vec, params);

        X_np1_vec = X_n_vec + (1/6) * (k1_vec + 2*k2_vec + 2*k3_vec + k4_vec);

        %---------------------------------------------------------------
        % MRP shadow-set switching
        % Assume the first 3 states are the MRP vector sigma_B/N
        %---------------------------------------------------------------
        sigma_BN_vec = X_np1_vec(1:3);

        if norm(sigma_BN_vec) > 1
            X_np1_vec(1:3) = -sigma_BN_vec / norm(sigma_BN_vec)^2;
        end

        %---------------------------------------------------------------
        % Save next state and current control
        %---------------------------------------------------------------
        X_vec_hist(:,n+1) = X_np1_vec;
        u_vec_hist(:,n) = u_n_vec;

    end

    % Store final control value
    u_vec_hist(:,N) = u_fun(t_hist(N), X_vec_hist(:,N), params);

end
Code
```

```
u_mission = @(t,X_vec, params_mission) attitude_pointing_control(t,X_vec,
    params_mission);
[X_vec_hist, u_vec_hist, t_hist] = integrate_mission(@f_T7, u_mission, SC.X0_vec,
    [0,6500], 1, params_mission);
sigma_BN_vec_300 = X_vec_hist(1:3, 301);
sigma_BN_vec_2100 = X_vec_hist(1:3, 2101);
sigma_BN_vec_3400 = X_vec_hist(1:3, 3401);
sigma_BN_vec_4400 = X_vec_hist(1:3, 4401);
sigma_BN_vec_5600 = X_vec_hist(1:3, 5601);
Submission
file = fopen(fullfile(outputFolder,'Task11_1.txt'), 'w');
fprintf(file, '%.7f␣%.7f␣%.7f', sigma_BN_vec_300);
fclose(file);

file = fopen(fullfile(outputFolder,'Task11_2.txt'), 'w');
fprintf(file, '%.7f␣%.7f␣%.7f', sigma_BN_vec_2100);
fclose(file);

file = fopen(fullfile(outputFolder,'Task11_3.txt'), 'w');
fprintf(file, '%.7f␣%.7f␣%.7f', sigma_BN_vec_3400);
fclose(file);

file = fopen(fullfile(outputFolder,'Task11_4.txt'), 'w');
fprintf(file, '%.7f␣%.7f␣%.7f', sigma_BN_vec_4400);
fclose(file);

file = fopen(fullfile(outputFolder,'Task11_5.txt'), 'w');
fprintf(file, '%.7f␣%.7f␣%.7f', sigma_BN_vec_5600);
fclose(file);

Mission Profile
figure('Position', [100, 100, 900, 300]);
hold on
grid on
box on

xlim([t_hist(1) t_hist(end)])

%-----------------------------------------
% Left axis: r_LMO2
%-----------------------------------------
yyaxis left
ylabel('$^N␣r_{LMO,2}$␣(km)','Interpreter','latex')
ylim([min(r_LMO2_hist) max(r_LMO2_hist)])
yl_left = ylim;

% Colors
c_sun  = [1.00 0.90 0.75];
c_sci  = [0.82 0.92 0.82];
c_comm = [0.90 0.90 0.90];

% Find switching indices
idx_change = [1, find(diff(mode_hist)~=0)+1, length(t_hist)];

%-----------------------------------------
% Colored intervals + labels
%-----------------------------------------
for j = 1:length(idx_change)-1
    i1 = idx_change(j);
```

```
    i2 = idx_change(j+1);

    t1 = t_hist(i1);
    t2 = t_hist(i2);

    if mode_hist(i1) == 1
        c = c_sun;   label_txt = 'Sun';
    elseif mode_hist(i1) == 2
        c = c_sci;   label_txt = 'Nadir';
    else
        c = c_comm;  label_txt = 'GMO';
    end

    patch([t1 t2 t2 t1], [yl_left(1) yl_left(1) yl_left(2) yl_left(2)], ...
          c, 'EdgeColor','none', 'FaceAlpha',0.45);

    % Label position (cleaner placement)
    if (t2 - t1) > 250
        t_mid = t1 + 0.5*(t2 - t1);   % slight right bias (fixes first Sun issue)
        y_txt = yl_left(1) + 0.5*(yl_left(2)-yl_left(1));

        text(t_mid, y_txt, label_txt, ...
            'HorizontalAlignment','center', ...
            'FontSize',10, ...
            'FontWeight','bold', ...
            'Color',[0.2 0.2 0.2]);
    end
end

    for j = 2:length(idx_change)-1
        t_sw = t_hist(idx_change(j))
        hLine = xline(t_sw, '--', 'Color', [0.35 0.35 0.35], 'LineWidth', 0.9);
        hLine.Annotation.LegendInformation.IconDisplayStyle = 'off';
    end

% Send patches to back
uistack(findobj(gca,'Type','patch'),'bottom')

%-----------------------------------------
% Plot signals
%-----------------------------------------
yyaxis left
h1 = plot(t_hist, r_LMO2_hist, '-', 'LineWidth',2, 'Color','k');
ax = gca;
ax.YColor = 'k';

yyaxis right
h2 = plot(t_hist, gamma_hist, '-.', 'LineWidth',2, 'Color','k');
ylabel('$\gamma$ (deg)','Interpreter','latex')
ax.YColor = 'k';

xlabel('$t$ (s)','Interpreter','latex')

%-----------------------------------------
% Legend (non-intrusive placement)
%-----------------------------------------
legend([h1 h2], ...
    {'$^N r_{LMO,2}$', '$\gamma$'}, ...
    'Interpreter','latex', ...
```

```
    'Location','southwest', ...   % <- key change
    'Box','off');
exportgraphics(gcf, 'Mission_0.pdf', 'ContentType','vector');
function add_mode_shading(t_hist, mode_hist, yl)

    hold on

    c_sun  = [1.00 0.90 0.75];
    c_sci  = [0.82 0.92 0.82];
    c_comm = [0.90 0.90 0.90];

    idx_change = [1, find(diff(mode_hist)~=0)+1, length(t_hist)];

    for j = 1:length(idx_change)-1
        i1 = idx_change(j);
        i2 = idx_change(j+1);

        t1 = t_hist(i1);
        t2 = t_hist(i2);

        if mode_hist(i1) == 1
            c = c_sun;
        elseif mode_hist(i1) == 2
            c = c_sci;
        else
            c = c_comm;
        end

        hPatch = patch([t1 t2 t2 t1], [yl(1) yl(1) yl(2) yl(2)], ...
            c, 'EdgeColor','none', 'FaceAlpha',0.30);
        hPatch.Annotation.LegendInformation.IconDisplayStyle = 'off';
    end

    for j = 2:length(idx_change)-1
        t_sw = t_hist(idx_change(j));
        hLine = xline(t_sw, '--', 'Color', [0.35 0.35 0.35], 'LineWidth', 0.9);
        hLine.Annotation.LegendInformation.IconDisplayStyle = 'off';
    end

    uistack(findobj(gca,'Type','patch'),'bottom')
end
SC Motion Plots
figure('Position', [100, 100, 900, 1200]); % [left, bottom, width, height]
tiledlayout(2,1)

nexttile
plot(t_hist, X_vec_hist(1,:), 'LineWidth', 1.2); hold on;
plot(t_hist, X_vec_hist(2,:), 'LineWidth', 1.2); hold on;
plot(t_hist, X_vec_hist(3,:), 'LineWidth', 1.2); hold on;
xlim([t_hist(1) t_hist(end)])
add_mode_shading(t_hist, mode_hist, [-1,1])
grid on;
ylabel('$\sigma_{{B/N,\;i}}$'); %xlabel('$t$ (s)');
legend('$\sigma_{{B/N,1}}$', '$\sigma_{{B/N,2}}$', '$\sigma_{{B/N,3}}$', 'Location', '
    eastoutside');

nexttile
plot(t_hist, X_vec_hist(4,:), 'LineWidth', 1.2); hold on;
plot(t_hist, X_vec_hist(5,:), 'LineWidth', 1.2); hold on;
```

```
plot(t_hist, X_vec_hist(6,:), 'LineWidth', 1.2); hold on;
xlim([t_hist(1) t_hist(end)])
grid on;
ylabel('$^B\omega_{{B/N,\;i}}$ (rad/s)'); xlabel('$t$ (s)');
legend('$^B\omega_{{B/N,1}}$', '$^B\omega_{{B/N,2}}$', '$^B\omega_{{B/N,3}}$', '
    Location', 'eastoutside');
add_mode_shading(t_hist, mode_hist, [-0.04,0.04])
exportgraphics(gcf, 'Mission_1.pdf', 'ContentType','vector');

Tracking Performance & Control
function [sigma_BR_vec_hist, B_w_BR_vec_hist, gamma_hist, r_LMO2_hist, mode_hist] =
    mission_analysis_hist(sigma_BN_vec_hist, B_w_BN_vec_hist, t_hist, params)
%=========================================================================
%
% Computes mission-related post-processed histories from integrated state
% histories.
%
% For each time step, this function computes:
%   1) attitude tracking error sigma_B/R
%   2) angular velocity tracking error B_w_B/R
%   3) gamma angle between LMO and GMO position vectors
%   4) second inertial component of LMO position
%
% Author: G. Montseny
% Date: April 20, 2026
%
%
% INPUT:                        Description                          Units
%
%  sigma_BN_vec_hist   -   MRP history B wrt N (3 x N)              -
%  B_w_BN_vec_hist     -   angular velocity history B wrt N (3 x N) rad/s
%  t_hist              -   time history (1 x N or N x 1)            s
%  params              -   mission/orbit parameter struct           -
%
% OUTPUT:
%
%  sigma_BR_vec_hist   -   MRP tracking error history (3 x N)       -
%  B_w_BR_vec_hist     -   angular velocity error history (3 x N)   rad/s
%  gamma_hist          -   gamma angle history (1 x N)              deg
%  r_LMO2_hist         -   2nd inertial component history of LMO    -
%
%=========================================================================

    % Ensure time is row vector
    t_hist = t_hist(:).';
    N = length(t_hist);

    % Preallocate
    sigma_BR_vec_hist = zeros(3, N);
    B_w_BR_vec_hist   = zeros(3, N);
    gamma_hist        = zeros(1, N);
    r_LMO2_hist       = zeros(1, N);
    mode_hist       = zeros(1, N);

    for k = 1:N

        %---------------------------------------------------------------
        % Extract current state
        %---------------------------------------------------------------
```

```
        sigma_BN_vec = sigma_BN_vec_hist(:,k);
        B_w_BN_vec   = B_w_BN_vec_hist(:,k);
        t_k          = t_hist(k);

        %---------------------------------------------------------------
        % Inertial geometry
        %---------------------------------------------------------------
        [N_rLMO_vec, ~] = inertial_state(params.LMO, t_k);
        [N_rGMO_vec, ~] = inertial_state(params.GMO, t_k);

        r_LMO = norm(N_rLMO_vec);
        r_GMO = norm(N_rGMO_vec);

        gamma_hist(k)  = acosd(N_rLMO_vec' * N_rGMO_vec / (r_LMO * r_GMO));
        r_LMO2_hist(k) = N_rLMO_vec(2);

        %---------------------------------------------------------------
        % Determine reference frame from mission logic
        %---------------------------------------------------------------
        if dot(N_rLMO_vec, [0;1;0]) > 0
            RN = N2Rs();
            N_w_RN_vec = [0;0;0];
            mode_hist(k) = 1; % Sun

        else
            gamma = gamma_hist(k);

            if gamma >= 35
                RN = N2Rn(params.LMO, t_k);
                N_w_RN_vec = inertial_w_RnN(params.LMO, t_k);
                mode_hist(k) = 2; % Science
            else
                RN = N2Rc(params, t_k);
                N_w_RN_vec = inertial_w_RcN(params, t_k);
                mode_hist(k) = 3; % Communication
            end
        end

        %---------------------------------------------------------------
        % Compute tracking errors
        %---------------------------------------------------------------
        [sigma_BR_vec, B_w_BR_vec] = track_err(sigma_BN_vec, B_w_BN_vec, RN,
            N_w_RN_vec);

        % Store
        sigma_BR_vec_hist(:,k) = sigma_BR_vec;
        B_w_BR_vec_hist(:,k)   = B_w_BR_vec;
    end

end

[sigma_BR_vec_hist, B_w_BR_vec_hist, gamma_hist, r_LMO2_hist, mode_hist] =
    mission_analysis_hist(X_vec_hist(1:3,:), X_vec_hist(4:6,:), t_hist, params_mission)
    ;

figure('Position', [100, 100, 900, 1200]); % [left, bottom, width, height]
tiledlayout(3,1)

nexttile
```

```
plot(t_hist, sigma_BR_vec_hist(1,:),   'LineWidth',1.2); hold on;
plot(t_hist, sigma_BR_vec_hist(2,:),   'LineWidth',1.2); hold on;
plot(t_hist, sigma_BR_vec_hist(3,:),   'LineWidth',1.2); hold on;
xlim([t_hist(1) t_hist(end)])
%plot(t_hist, vecnorm(X_vec_hist(1:3,:))); hold on;
add_mode_shading(t_hist, mode_hist, [-1,1])
grid on;
ylabel('$\sigma_{{B/R_c,\;i}}$'); %xlabel('$t$ (s)');
legend('$\sigma_{{B/R_c,1}}$', '$\sigma_{{B/R_c,2}}$', '$\sigma_{{B/R_c,3}}$',  '
    Location', 'eastoutside');

nexttile
plot(t_hist, B_w_BR_vec_hist(1,:),   'LineWidth',1.2); hold on;
plot(t_hist, B_w_BR_vec_hist(2,:),   'LineWidth',1.2); hold on;
plot(t_hist, B_w_BR_vec_hist(3,:),   'LineWidth',1.2); hold on;
xlim([t_hist(1) t_hist(end)])
add_mode_shading(t_hist, mode_hist, [-0.04,0.04])
grid on;
ylabel('$^B\omega_{{B/R_c,\;i}}$␣(rad/s)'); %xlabel('$t$ (s)');
legend('$^B\omega_{{B/R_c,1}}$', '$^B\omega_{{B/R_c,2}}$', '$^B\omega_{{B/R_c,3}}$', '
    Location', 'eastoutside');


nexttile
plot(t_hist, u_vec_hist(1,:),   'LineWidth',1.2); hold on;
plot(t_hist, u_vec_hist(2,:),   'LineWidth',1.2); hold on;
plot(t_hist, u_vec_hist(3,:),   'LineWidth',1.2); hold on;
xlim([t_hist(1) t_hist(end)])
add_mode_shading(t_hist, mode_hist, [-0.005,0.005])
ylim([-0.005,0.005]);
grid on;
ylabel('$^Bu_i$␣(kgm$^2$s$^{-2}$)'); xlabel('$t$␣(s)');
legend('$^Bu_1$','$^Bu_2$', '$^Bu_3$',  'Location', 'eastoutside');


exportgraphics(gcf, 'Mission_2.pdf', 'ContentType','vector');
```